Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Quantifying the Approximation Error of Task-Vector Merging in the Convex Hull

Bachelor's Thesis

Robin Koppelhuber

2026-04-16

Advisors: Prof. Dr. Fanny Yang, Javier Abad Martinez
Department of Computer Science, ETH Zürich

Abstract

Model merging methods combine the parameters of multiple fine-tuned models into a single multi-task model without retraining. In the pretrain-then-finetune paradigm, the difference between each expert's parameters and the shared initialization is called a task vector. A dominant class of merging methods applies heuristics to these vectors and searches for a merged model within their convex hull, where the convex hull has a dimension of at most the number of tasks minus one. Prior work has shown that useful trade-offs exist in this convex set, but the approximation error incurred by restricting the search to such a low-dimensional subset has not been quantified. We compare two merging methods (Task Arithmetic and TIES-Merging), constrained to the convex set, against two training-based multi-objective optimization methods (augmented Chebyshev scalarization and EPO Search) that optimize over the full parameter space. All methods are evaluated on binary GLUE tasks using RoBERTa-base, in both a four-task and a focused two-task setting selected for high inter-task interference. Training-based methods consistently achieve trade-offs unattainable within the convex set. Task-vector averaging degrades catastrophically under destructive interference, while TIES-Merging mitigates this but retains a persistent gap to training-based methods. On the two-task benchmark, TIES-Merging's Pareto front is clearly dominated despite dense search over merging coefficients, indicating that the gap is a geometric property of the convex set rather than a failure to find optimal coefficients. These findings suggest that in overparameterized multi-task settings, the convex hull of task vectors is a fundamentally limited model class that excludes trade-offs accessible to gradient-based optimization over the full parameter space. We discuss the limitations of our single-architecture, binary-task setup and outline directions for extending this analysis.[1]

[1]Source code available at https://github.com/robin-koppelhuber/Bachelor-Thesis-SML

Contents

Contents ii

1 Introduction **1**
1.1 Settings 2
1.2 Design Space 3
1.3 Related Approaches 5

2 Preliminaries **8**
2.1 Formal Setting and Notation 8
2.2 Reference Methods 11
2.2.1 Chebyshev scalarization 11
2.2.2 EPO Search 13
2.3 Merging Methods 16
2.3.1 Task Arithmetic 16
2.3.2 TIES-Merging 17
2.3.3 Weight Interpolation as a Steerable Method 17
2.4 Loss Landscape Geometry 19

3 Experiments **23**
3.1 Methodology 24
3.1.1 Model and Head Architecture 24
3.1.2 Tasks and Datasets 25
3.1.3 Data Splits 26
3.1.4 Evaluation Metrics and Normalization 27
3.1.5 Expert Models and Cross-Evaluation 28
3.1.6 Training-Based Methods: Implementation 29
3.1.7 Merging Methods: Implementation 30
3.1.8 Preference Vectors 30
3.2 Results 31
3.2.1 Calibration: Single-Task Recovery 31

3.2.2 Four-Task Benchmark 32
3.2.3 Two-Task Benchmark: MRPC–QNLI 34
3.3 Limitations 36

4 Conclusion and Future Work **39**
4.1 Conclusion 39
4.2 Future Work 40

Bibliography **43**

A MOO formalizations **50**
A.1 Pathological MOL example 50
A.2 Augmented Weighted Chebyshev Scalarization: Proof and Discussion 52

B Experiment Hyperparameters **54**

C Additional results **56**

Chapter 1

Introduction

The defining property of model merging (MM) methods is the combination of the parameters of multiple machine learning models to obtain a single model for inference. In a simple setting, one may think about multiple models instantiated from the same architecture and collecting the parameters of these models into a vector, one for each model. A model merging method can then be thought of as a function mapping these parameter vectors and additional arguments to a final merged model of the same architecture.

The most influential factor in the design space of model merging methods is the motivation and setting in which a method is used. Before exploring the design space of these methods, we introduce some preliminary concepts. In recent years, large model training has been broadly split into pretraining and post-training, though a more nuanced description of the modern training pipeline exists [1]. Methods papers often refer to these stages as the 'time' when they intervene, such as training time, post-training time, and inference time. While there is some ambiguity in the literature about which stage of the training pipeline is referred to as fine-tuning, we will define it as any post-pretraining stage that specializes a model toward a specific task or objective.

Model merging, as defined above, emerged alongside the rise of open-weight machine learning models featuring a pretraining and post-training split, demonstrated on models such as CLIP [2], BERT [3], and T5 [4]. The pretrain-then-finetune paradigm enabled practitioners to specialize a single pretrained model toward multiple tasks or objectives by fine-tuning, yielding a collection of expert models sharing a common initialization. This proliferation of fine-tuned model variants, widely distributed on community platforms such as Hugging Face, motivated the development of computationally inexpensive methods to combine their capabilities into a single model without incurring the cost of retraining from scratch.

Model merging is motivated by several complementary goals. First, it enables the reuse of existing fine-tuned artifacts, combining capabilities from multiple expert models into a single model without retraining. Second, it offers inference efficiency: unlike model ensembles or mixtures of experts (see section 1.3), a merged model requires only a single forward pass at inference time, with no increase in computational cost relative to a single model. Third, in the federated learning setting [5], model merging serves as a key tool to merge multiple locally trained models on private data, though the shared model weights may themselves be vulnerable to data extraction attacks [6]. Finally, model merging can be used to mitigate catastrophic forgetting [7]. Beyond these primary motivations, model merging has also been shown to yield additional favorable properties, such as improved pre-training stability through convergence to flatter minima [8], as well as better performance on out-of-distribution generalization. For a comprehensive list of motivations, see [9].

1.1 Settings

There are three primary axes along which to classify model merging settings, which dictate the possible design decisions

Data access When models are shared on community platforms, the original training data is often not published alongside the weights. As we will discuss, knowledge of the training data, or suitable proxies such as the diagonal approximation of the Fisher information matrix of the model parameters with respect to the training loss [10], can be crucial for avoiding parameter interference during merging. We refer to methods that operate without any such data access as **data-free** [11, 12]. At the other extreme, **data-aware** methods assume full access to the training data, enabling a more informed merging process [13]. Between these two poles, a third class of methods uses surrogate data, labeled or unlabeled, to approximate the information provided by the original training data [14].

Multi-Task Learning (MTL) vs. Multi-Objective Learning (MOL) : In the canonical MTL setting [15, 16], we are given multiple task-specific data distributions, each with an associated loss function, and the goal is to find a single model minimizing a weighted combination of the per-task losses. The canonical MOL setting instead assumes a single data distribution and multiple objectives to be optimized simultaneously, yielding a Multi-Objective Optimization (MOO) problem. MTL problems can be cast as MOL problems [17] by treating the performance on each task as a separate objective. However, MTL should be considered a strict subset of MOL. Since MTL tasks are evaluated independently

(each input is assessed under only one task's loss), a sufficiently expressive model can, in principle, achieve near optimal performance on all tasks simultaneously, which suggests that MTL objectives are not fundamentally conflicting. This claim is supported empirically by [18] and further discussed in Remark 2.3. In practice, however, finding such a model via gradient-based optimization remains challenging due to gradient conflicts between tasks and limited model capacity. In contrast, MOL encompasses settings where objectives are conflicting by construction, and no increase in model capacity resolves the fundamental Pareto tradeoff (see section A.1 for a pathological example). Some authors distinguish meta-tasks in MTL, where tasks in the context of LLMs could refer to coding, mathematics, and translation, though the MTL framework does not require such distinctions. For multiple objectives in the LLM context, one may think of quantities such as privacy, safety, fairness, harmlessness, helpfulness, honesty, verbosity, or instruction following.

shared initialization vs. unrelated models In the most common setting, the models to be merged were fine-tuned from a shared pretrained model. In such cases, several favorable properties of the loss landscape with respect to the model parameters emerge (section 2.4), widely considered a key enabling factor for effective model merging. If the models share the same architecture but were fine-tuned from different pretrained models, these properties are harder to obtain. A fundamental obstacle is the permutation invariance of neural networks: in a linear layer, any permutation of the output neurons yields a functionally identical network, as the corresponding weight matrices can be adjusted accordingly. Two independently trained models may therefore have learned equivalent functions but with entirely different neuron orderings, rendering naive parameter averaging semantically meaningless. Methods such as Git Re-Basin [19] address this by solving a linear assignment problem to find an optimal permutation that aligns the weight spaces of the two models before merging. To which extent the favorable properties of the loss landscape hold under permutation invariance has been studied by [20]. However, since model merging is most naturally motivated in the pretrain-then-finetune paradigm, where a shared initialization is given by construction, we do not discuss these methods further. For a survey on such methods, see [21].

1.2 Design Space

To further motivate our contribution, we will now introduce the different axes of the model merging method design space.

intervention time An important property of a merging method is at which stage of the training pipeline it acts. Methods exist for every stage, from pretraining through post-training to inference time [9]. Importantly, a method acting at a training stage does not imply that it is optimization-based, as demonstrated for example by Pre-trained Model Average (PMA) [8], which merges pretraining checkpoints using moving-window weight averaging in a data-free, optimization-free manner. However, intervening early in the pipeline can benefit the merging process, for instance by constraining fine-tuned model weights to lie in orthogonal subspaces [22, 23] or by continuously merging models during fine-tuning [13]. Such methods come with their own drawbacks. First, when multiple expert models are jointly optimized toward a shared merging objective, they may develop implicit mutual dependencies, reducing the effective specialization of each expert, a phenomenon we are not aware of having been formally studied in the literature. Second, methods that modify the fine-tuning procedure no longer cater to the original motivation of model merging, namely the reuse of existing fine-tuned artifacts without access to or modification of the fine-tuning process.

Steerability Some merging methods produce a single fixed merged model, while others accept a preference vector over tasks or objectives as input, enabling the user to navigate the Pareto front of trade-offs between them. Non-steerable methods, e.g. methods that determine merging coefficients from auxiliary data [14], produce a single model whose trade-off is implicitly fixed by the method. Steerable methods, in contrast, expose the preference as an explicit input, producing a different merged model for each point on the trade-off frontier.

Marginal cost of preference following For steerable methods, an important practical consideration is the marginal cost of producing a merged model for a new preference vector after the first. At one extreme, methods that are both optimization-free and data-free often incur negligible costs per preference and require no upfront investment, making the notion of amortization less meaningful. At the other extreme, some methods invest significant upfront computation, for instance by training a hypernetwork [24] or a preference-conditioned parametric model [25], which can subsequently generate merged models for any preference vector at minimal marginal cost, even at inference time.

Optimization-free vs. optimization-based We consider a method optimization-free if it applies closed-form or heuristic operations directly to the model parameters, requiring no iterative updates. Optimization-based methods in contrast employ an iterative procedure to optimize some objective related to merged model performance, ranging from gradient-

based methods to evolutionary search. Crucially, this axis is independent of the data access axis: optimization-based methods need not be data-aware, as demonstrated by [25]. Conversely, data-aware methods need not be optimization-based [10].

Large sub-categories of methods reveal additional design choices that will be discussed there.

1.3 Related Approaches

Model merging occupies a specific niche in the space of methods for combining knowledge from multiple models or tasks. To situate merging relative to its alternatives, we organize the discussion around the primary resource constraints and desiderata that determine method selection in practice: inference cost, data requirements, and the computational budget available for producing the combined model.

Ensembles and logit-level combination. The most direct alternative to weight-level merging is prediction-level combination. Deep ensembles [26] aggregate the outputs of independently trained models, typically by averaging predicted probabilities or logits. Ensembles consistently outperform individual models and, by extension, weight-averaged models [27], while additionally providing calibrated uncertainty estimates. The cost is linear scaling of inference compute and memory with the number of ensemble members, since each input requires a full forward pass through every model. When inference cost is not a binding constraint, for instance in offline batch processing or safety-critical applications where uncertainty quantification is essential, ensembles are generally preferable to model merging. Stacking [28] and other learned combination rules over model outputs offer further flexibility by training a meta-learner on the predictions of base models, though at the cost of requiring a held-out dataset for the meta-learner. Logit-level ensembles can also be viewed as a special case of output-space merging, contrasting with the parameter-space merging that is the focus of this work.

Mixture-of-experts and conditional computation. Mixture-of-experts (MoE) architectures [29, 30] route each input to a subset of specialized sub-networks (experts) via a learned gating function, activating only a fraction of the total parameters per forward pass. This conditional execution decouples total model capacity from per-input inference cost, achieving a trade-off unavailable to both ensembles (which activate all parameters) and merged models (which compress all expertise into a single parameter set). Modern MoE variants such as the Switch Transformer [31] scale to hundreds of billions of parameters while maintaining manageable per-token compute. However, MoE requires either joint training of experts and gating, or post-hoc router

training given existing experts [32], both of which demand data access and non-trivial engineering. MoE is therefore preferable to model merging when the practitioner controls the training pipeline and requires high per-task performance without full ensemble cost, but is less suitable when the goal is to reuse independently trained artifacts without modification.

Knowledge distillation. Knowledge distillation [33] trains a student model to mimic the outputs of one or more teacher models. Multi-teacher distillation [34] extends this to the setting most relevant to model merging: given T expert teachers, a single student is trained to reproduce the teachers' predictions across all tasks. Unlike model merging, distillation can in principle recover the full predictive power of the teacher ensemble in a single student network, but it requires access to training data (or suitable proxy data) and incurs a full training cycle for the student. Distillation is therefore the method of choice when data is available and a high-quality single model is the end goal, particularly when the student architecture can be made smaller than the teachers, yielding both capability combination and model compression [35].

Hypernetworks. Hypernetworks [36] parameterize a mapping from a conditioning signal (e.g., a preference vector or task descriptor) to the weights of a target network. In the multi-objective setting, [24] train a hypernetwork to generate Pareto-optimal models for any user-specified preference at inference time, amortizing the cost of Pareto front exploration into a single upfront training phase. This yields steerable, low marginal-cost multi-objective models, but requires substantial initial training and architectural overhead. Hypernetworks are preferable when the Pareto front must be queried repeatedly at diverse preferences and training compute is available upfront; model merging is preferable when the goal is rapid, optimization-free combination of existing artifacts.

Compression methods. When the primary goal is inference efficiency rather than capability combination, model compression via pruning [37], quantization [38], or structured sparsification offers a more direct path. These methods reduce the computational and memory footprint of a single model without combining information from multiple sources. They are orthogonal to model merging and can in principle be applied to the output of any merging procedure, though interactions between merging and subsequent compression have not been extensively studied.

Summary. Model merging is best understood as the method of last resort when stronger alternatives are infeasible or low cost approximate Steerability is required: in its simplest setting, it requires no data, no training, and

no modification of the expert models, at the cost of restricting the merged model to a low-dimensional subspace of parameter space. When inference cost is not a binding constraint, ensembles will generally outperform a merged model. When data, compute, or control over the training pipeline is available, distillation, MoE, or direct multi-task training will generally yield superior models at any single operating point. However, the approximation error incurred by the restriction to this low-dimensional subspace has not been quantified. The central question of this thesis is whether restricting the search to the convex hull of (preprocessed) task vectors incurs a significant approximation error relative to gradient-based optimization over the full parameter space, and if so, whether the gap reflects a failure to find optimal merging coefficients or a structural limitation of the convex set itself.

Chapter 2

Preliminaries

2.1 Formal Setting and Notation

We now introduce a formal multi-task learning setting, cast it as a multi-objective optimization problem following [17], and discuss scalarizations following [39].

Per-Task Supervised Learning

Let T denote the number of tasks and write $[N] = \{1, \dots, N\}$ for $N \in \mathbb{N}$. For each task $t \in [T]$, let $\mathcal{X}^t$ be the input space, $\mathcal{Y}^t$ the label space, and P^t a probability distribution over $\mathcal{X}^t \times \mathcal{Y}^t$. We assume throughout that the task input supports are pairwise disjoint. In practice, this is satisfied whenever the input encodes task identity, for instance, through distinct input formats, task-specific prefixes, or prompt templates in language models. Formally, $\mathrm{Supp}(P^t_{\mathcal{X}}) \cap \mathrm{Supp}(P^{t'}_{\mathcal{X}}) = \emptyset$ for $t \neq t'$, where $P^t_{\mathcal{X}}$ denotes the marginal of P^t on $\mathcal{X}^t$ (see Remark 2.3 for a discussion of this assumption). Let $\mathcal{D}^t = (x^t_i, y^t_i)_{i=1}^{N_t} \overset{\text{i.i.d.}}{\sim} P^t$ be N_t samples from P^t, and let $\ell^t \colon \mathcal{Y}^t \times \mathcal{Y}^t \to \mathbb{R}_{\geq 0}$ be the loss function of task t.

Let $\mathcal{H}^t_{\text{all}}$ denote the set of all measurable functions from $\mathcal{X}^t$ to $\mathcal{Y}^t$. The **population risk** of a hypothesis $h \in \mathcal{H}^t_{\text{all}}$ on task t is

$$\mathcal{R}^t \colon \mathcal{H}^t_{\text{all}} \to \mathbb{R}_{\geq 0}, \quad h \mapsto \mathbb{E}_{(X,Y)\sim P^t}\big[\ell^t\big(h(X), Y\big)\big].$$

The **empirical risk** of h on $\mathcal{D}^t$ is

$$\hat{\mathcal{R}}^t \colon \mathcal{H}^t_{\text{all}} \to \mathbb{R}_{\geq 0}, \quad h \mapsto \frac{1}{N_t} \sum_{i=1}^{N_t} \ell^t\big(h(x^t_i), y^t_i\big).$$

The **Bayes risk** for task t is $\mathcal{R}^t_\star = \inf_{h \in \mathcal{H}^t_{\text{all}}} \mathcal{R}^t(h)$, and the **excess risk** of $h \in \mathcal{H}^t_{\text{all}}$ is

$$\mathcal{E}^t \colon \mathcal{H}^t_{\text{all}} \to \mathbb{R}_{\geq 0}, \quad h \mapsto \mathcal{R}^t(h) - \mathcal{R}^t_\star.$$

Finally, the **approximation error** of a hypothesis subset $\mathcal{H} \subseteq \mathcal{H}^t_{\text{all}}$ on task t is

$$\mathcal{E}^t_{\text{appr}} \colon 2^{\mathcal{H}^t_{\text{all}}} \to \mathbb{R}_{\geq 0}, \quad \mathcal{H} \mapsto \inf_{h \in \mathcal{H}} \mathcal{R}^t(h) - \mathcal{R}^t_\star = \inf_{h \in \mathcal{H}} \mathcal{E}^t(h).$$

Parameterized Model Class

In practice, we do not optimize over $\mathcal{H}^t_{\text{all}}$ but over a parameterized model class. Let $\Theta \subseteq \mathbb{R}^d$ be a parameter space. For each task $t \in [T]$, let

$$f^t \colon \Theta \times \mathcal{X}^t \to \mathcal{Y}^t, \quad (\theta, x) \mapsto f^t(\theta, x)$$

denote the model's prediction on task t, parameterized by $\theta \in \Theta$.[1] With a standard abuse of notation, we write $\mathcal{R}^t(\theta)$ for $\mathcal{R}^t(f^t(\theta, \cdot))$ and $\mathcal{E}^t(\theta)$ for $\mathcal{E}^t(f^t(\theta, \cdot))$, overloading the domain of $\mathcal{R}^t$ and $\mathcal{E}^t$ from $\mathcal{H}^t_{\text{all}}$ to Θ. Analogously, we define the approximation error of a parameter subset $\Theta' \subseteq \Theta$ on task t as

$$\mathcal{E}^t_{\text{appr}}(\Theta') = \inf_{\theta \in \Theta'} \mathcal{R}^t(\theta) - \mathcal{R}^t_\star.$$

Note that $\mathcal{E}^t_{\text{appr}}(\Theta') \geq 0$ by definition, and $\mathcal{E}^t_{\text{appr}}(\Theta') \geq \mathcal{E}^t_{\text{appr}}(\Theta)$ for any $\Theta' \subseteq \Theta$, i.e., restricting the parameter set can only increase the approximation error.

Multi-Objective Optimization

Rather than optimizing a single scalarized loss, we follow [17] and treat the T population risks as a vector-valued objective

$$\mathbf{R} \colon \Theta \to \mathbb{R}^T_{\geq 0}, \quad \theta \mapsto \big(\mathcal{R}^1(\theta), \ldots, \mathcal{R}^T(\theta)\big),$$

yielding a multi-objective optimization (MOO) problem $\min_{\theta \in \Theta} \mathbf{R}(\theta)$, where the minimum is understood in the sense of Pareto optimality.

Definition 2.1 (Pareto Dominance) *Let $\theta, \theta' \in \Theta$.*

1. *θ **weakly dominates** θ', written $\theta \preceq \theta'$, if $\mathcal{R}^t(\theta) \leq \mathcal{R}^t(\theta')$ for all $t \in [T]$.*
2. *θ **strictly dominates** θ', written $\theta \prec \theta'$, if $\theta \preceq \theta'$ and $\mathbf{R}(\theta) \neq \mathbf{R}(\theta')$.*

Definition 2.2 (Pareto Optimality and Pareto Front) *Let $\Theta' \subseteq \Theta$.*

1. *A parameter $\theta^\star \in \Theta'$ is **weakly Pareto optimal** in Θ' if there is no $\theta \in \Theta'$ with $\mathcal{R}^t(\theta) < \mathcal{R}^t(\theta^\star)$ for all $t \in [T]$ simultaneously.*

[1] In architectures with task-specific heads (e.g., fine-tuned encoder models), θ collects both the shared backbone parameters and the task-specific head parameters. In architectures without explicit task-specific parameters (e.g., autoregressive language models), θ is the full parameter vector, $f^t = f^{t'}$ for all t, t', and task routing is achieved implicitly through the input encoding.

2. *A parameter $\theta^\star \in \Theta'$ is **Pareto optimal** in Θ' if no $\theta \in \Theta'$ strictly dominates $\theta^\star$.*
3. *The **Pareto set** of Θ' is $\mathrm{PS}(\Theta') = \{\theta \in \Theta' : \theta \text{ is Pareto optimal in } \Theta'\}$.*
4. *The **Pareto front** of Θ' is the image $\mathrm{PF}(\Theta') = \mathbf{R}\big(\mathrm{PS}(\Theta')\big) \subseteq \mathbb{R}^T_{\geq 0}$.*

Every Pareto optimal point is weakly Pareto optimal, but the converse does not hold: a weakly Pareto optimal point may be strictly dominated, i.e., there may exist $\theta \in \Theta'$ with $\mathcal{R}^t(\theta) \leq \mathcal{R}^t(\theta^\star)$ for all t and $\mathcal{R}^t(\theta) < \mathcal{R}^t(\theta^\star)$ for at least one t.

Remark 2.3 (MTL vs. MOL and Collapse of the Pareto Front) *In the multi-task learning (MTL) setting formalized above, each task t is associated with its own data distribution P^t, and for a given sample $(x, y) \sim P^t$ the model is evaluated only under loss ℓ^t. No single sample is simultaneously evaluated under multiple loss functions. The multi-objective structure arises solely from the shared dependence of all per-task population risks on the common parameters θ.*

In contrast, in a multi-objective learning (MOL) setting, a single distribution P generates samples that are simultaneously evaluated under all objectives $\ell^1, \ldots, \ell^T$: for each input, the model produces one output that must satisfy potentially conflicting criteria (e.g., helpfulness vs. harmlessness in language model alignment).

The disjoint support assumption is the formal expression of this distinction. It is satisfied by construction whenever the input encodes task identity, for instance through distinct input formats, task-specific prefixes, or prompt templates, as is standard in modern multi-task systems. This assumption has consequences for the Pareto front. Since the supports are disjoint, a sufficiently expressive model can in principle achieve near-optimal risk on each task's input support independently. That is, if the parameterized model class is rich enough, there exists $\theta^\star \in \Theta$ such that $\mathcal{R}^t(\theta^\star) \approx \mathcal{R}^t_\star$ for all $t \in [T]$ simultaneously, and the Pareto front approximately collapses to a single point. This is a standard consequence of the disjoint support structure: the model need not compromise between tasks because no input is evaluated under multiple losses. In the MOL setting, this collapse need not occur: individual samples may admit no output simultaneously optimal under all objectives, yielding a non-degenerate Pareto front regardless of model capacity (see section A.1).

Our experiments are in the MTL setting. Any observed Pareto trade-offs are therefore attributable to limited model capacity, optimization limitations, or restricted model classes, not to fundamental objective conflict.

Utopia Point, Nadir Point, and Normalization

Definition 2.4 (Utopia and Nadir Points) *Let $\Theta' \subseteq \Theta$.*

1. *The **utopia point** (or ideal point) of Θ' is $\mathbf{z}^\star(\Theta') \in \mathbb{R}^T$ with components $z_t^\star(\Theta') = \inf_{\theta\in\Theta'} \mathcal{R}^t(\theta)$. It represents the best achievable risk on each task independently.*
2. *The **nadir point** of Θ' is $\mathbf{z}^{\mathrm{nad}}(\Theta') \in \mathbb{R}^T$ with components $z_t^{\mathrm{nad}}(\Theta') = \sup_{\theta\in\mathrm{PS}(\Theta')} \mathcal{R}^t(\theta)$. It represents the worst risk on each task attained by any Pareto-optimal solution.*

The utopia point is generally not attainable: if $\mathbf{z}^\star \in \mathrm{PF}(\Theta')$, the Pareto front is a singleton and the MOO problem is trivial (as may occur in MTL with sufficient capacity, cf. Remark 2.3). Together, $\mathbf{z}^\star$ and $\mathbf{z}^{\mathrm{nad}}$ bound the Pareto front componentwise [39].

Assuming $z_t^\star < z_t^{\mathrm{nad}} < +\infty$ for all $t \in [T]$, we define the **normalized risk** componentwise as

$$\tilde{\mathcal{R}}^t \colon \Theta \to \mathbb{R}, \quad \theta \mapsto \frac{\mathcal{R}^t(\theta) - z_t^\star}{z_t^{\mathrm{nad}} - z_t^\star}.$$

This normalization maps the Pareto front into $[0,1]^T$ and ensures that the scalarizations introduced in section 2.2 are not dominated by objectives with larger absolute scale [39, Part I, Chapter 2.4]. Importantly, it preserves the Pareto ordering: since $a_t := 1/(z_t^{\mathrm{nad}} - z_t^\star) > 0$ and $b_t := -z_t^\star/(z_t^{\mathrm{nad}} - z_t^\star)$ are constants with $a_t > 0$, the transformation $\tilde{\mathcal{R}}^t(\theta) = a_t\mathcal{R}^t(\theta) + b_t$ is a positive affine transformation applied independently to each component. If $\theta \prec \theta'$ holds under $\mathbf{R}$, then $\mathcal{R}^t(\theta) \leq \mathcal{R}^t(\theta')$ for all t, with strict inequality for some t, by definition. Since scaling with $a_t > 0$ and adding b_t preserves both weak and strict inequalities, $\tilde{\mathcal{R}}^t(\theta) \leq \tilde{\mathcal{R}}^t(\theta')$ for all t with strict inequality for the same t, so $\theta \prec \theta'$ under $\tilde{\mathbf{R}}$. The reverse implication follows identically from the invertibility of the transformation. Therefore, $\mathrm{PS}(\Theta')$ is invariant under the normalization.

2.2 Reference Methods

We first discuss two training-based preference-aware MOO methods, against which we will compare the model merging methods in section 2.3.

2.2.1 Chebyshev scalarization

Scalarization methods reduce the vector-valued problem $\min_{\theta\in\Theta'} \mathbf{R}(\theta)$ to a family of scalar optimization problems, parameterized by a preference vector. Different choices of the preference yield different points on (or near) the Pareto front. We present three scalarizations of increasing strength, following the exposition in [39]. Throughout, let $\Theta' \subseteq \Theta$ be the feasible parameter

set, let $\mathbf{z}^\star = \mathbf{z}^\star(\Theta')$ denote the utopia point as in Definition 2.4, and let $\Delta^{T-1} = \{\mathbf{w} \in \mathbb{R}^T_{>0} : \sum_{t=1}^T w_t = 1\}$ denote the open $(T-1)$-simplex.[2]

Definition 2.5 (Linear Scalarization) *For $\mathbf{w} \in \Delta^{T-1}$, the **linear scalarization** (or weighted sum method) is the problem*

$$\min_{\theta \in \Theta'} \sum_{t=1}^{T} w_t \, \mathcal{R}^t(\theta).$$

Every minimizer of the linear scalarization is Pareto optimal [39, Part II, Theorem 3.1.2]. However, only Pareto optimal points on convex regions of the Pareto front can be obtained as minimizers; points on non-convex regions are inaccessible to any linear scalarization [39, Part II, Chapter 3.1]. This limitation motivates the following alternative.

Definition 2.6 (Weighted Chebyshev Scalarization) *For $\mathbf{w} \in \Delta^{T-1}$ and a reference point $\mathbf{z} \in \mathbb{R}^T$ with $z_t \leq \mathcal{R}^t(\theta)$ for all $t \in [T]$ and all $\theta \in \Theta'$ (e.g., $\mathbf{z} = \mathbf{z}^\star$), the **weighted Chebyshev scalarization** (or weighted Tchebycheff method) is the problem*

$$\min_{\theta \in \Theta'} \max_{t \in [T]} \; w_t\big(\mathcal{R}^t(\theta) - z_t\big).$$

Unlike the linear scalarization, the weighted Chebyshev scalarization can access all Pareto optimal points, including those on non-convex regions of the front. More precisely, every minimizer $\theta^\star \in \Theta'$ of the above problem is weakly Pareto optimal, and if $\theta^\star$ is the unique minimizer, it is Pareto optimal. Conversely, every Pareto optimal parameter vector can be obtained as the solution to the above problem for some $w \in \Delta^{T-1}$ [39, Part II, Theorems 3.4.2, 3.4.5 and Corollary 3.4.4]. However, when the minimizer is not unique, some minimizers may be only weakly Pareto optimal. To guarantee (strict) Pareto optimality, the scalarization is augmented with a strictly monotone term.

Definition 2.7 (Augmented Weighted Chebyshev Scalarization) *For $\mathbf{w} \in \Delta^{T-1}$, a reference point $\mathbf{z}$ as in Definition 2.6, and an augmentation parameter $\rho > 0$, the **augmented weighted Chebyshev scalarization** is the problem*

$$\min_{\theta \in \Theta'} \left[\max_{t \in [T]} \; w_t\big(\mathcal{R}^t(\theta) - z_t\big) \; + \; \rho \sum_{t=1}^{T} w_t\big(\mathcal{R}^t(\theta) - z_t\big) \right].$$

This is a weighted variant of the augmented weighted Tchebycheff problem [39, Part II, Problem (3.4.5)], which uses unweighted absolute deviations

[2] We normalize preference vectors to sum to one for interpretability. The scalarization theorems in [39] are stated for arbitrary $\mathbf{w} \in \mathbb{R}^T_{>0}$; since the Chebyshev objective is positively homogeneous in $\mathbf{w}$, restricting to Δ^{T-1} is without loss of generality.

$\rho \sum_t |\mathcal{R}^t(\theta) - z_t|$ in the augmentation term. Our variant places the preference weights $\mathbf{w}$ in the augmentation, coupling the strictly monotone correction with the Chebyshev term.

Proposition 2.8 (Pareto optimality of minimizers) *Let $\Theta' \subseteq \Theta$, $\mathbf{w} \in \Delta^{T-1}$, $\rho > 0$, and $\mathbf{z} \in \mathbb{R}^T$ with $z_t \leq \mathcal{R}^t(\theta)$ for all $t \in [T]$ and $\theta \in \Theta'$. Then every minimizer of the augmented weighted Chebyshev scalarization (Definition 2.7) over Θ' is Pareto optimal in Θ'.*

Proof See section A.2. □

Without the augmentation ($\rho = 0$), minimizers of the weighted Chebyshev scalarization (Definition 2.6) are only guaranteed to be weakly Pareto optimal [39, Part II, Theorem 3.4.2]. The augmentation upgrades this to full Pareto optimality: any strict improvement on a single objective, holding others fixed, strictly decreases the augmentation term and hence the overall objective.

Remark 2.9 (Attainability of Pareto optimal points) *For the uncoupled formulation (Problem (3.4.5) in [39]), [39, Part II, Theorem 3.4.6] establishes that every properly Pareto optimal point can be obtained as a minimizer for some $\mathbf{w}$ and sufficiently small ρ. Our formulation can be reduced to an instance of Problem (3.4.5) via the transformed objectives $g_t(\theta) = w_t(\mathcal{R}^t(\theta) - z_t)$, but the converse does not transfer directly; see section A.2 for details. For our purposes, Theorem 2.8 suffices.*

Application to normalized objectives. When per-task risks differ in scale, a uniform $\mathbf{w}$ does not correspond to equal importance. We therefore apply the augmented Chebyshev scalarization to the normalized risk $\tilde{\mathcal{R}}^t$ from section 2.1, with reference point $\tilde{\mathbf{z}}^\star = \mathbf{0}$ (which is the utopia point of the normalized problem, since $\tilde{\mathcal{R}}^t(\theta) \geq 0$ for all $\theta \in \Theta'$ and $\inf_{\theta \in \Theta'} \tilde{\mathcal{R}}^t(\theta) = 0$ by construction):

$$\min_{\theta \in \Theta'} \left[\max_{t \in [T]} \; w_t \, \tilde{\mathcal{R}}^t(\theta) \; + \; \rho \sum_{t=1}^{T} w_t \, \tilde{\mathcal{R}}^t(\theta) \right]. \tag{2.1}$$

Since this is an instance of Definition 2.7 applied to the normalized objectives $\tilde{\mathbf{R}}$ with reference point $\tilde{\mathbf{z}}^\star = \mathbf{0}$, the same theoretical guarantees apply. Since $\mathrm{PS}(\Theta')$ is invariant under the normalization (cf. section 2.1), these are also exactly the Pareto optimal points with respect to the original objectives $\mathbf{R}$. For a visualization see Figure 2.1.

2.2.2 EPO Search

The augmented Chebyshev scalarization reduces the MOO problem to a family of single-objective problems, one per preference vector. An alternative

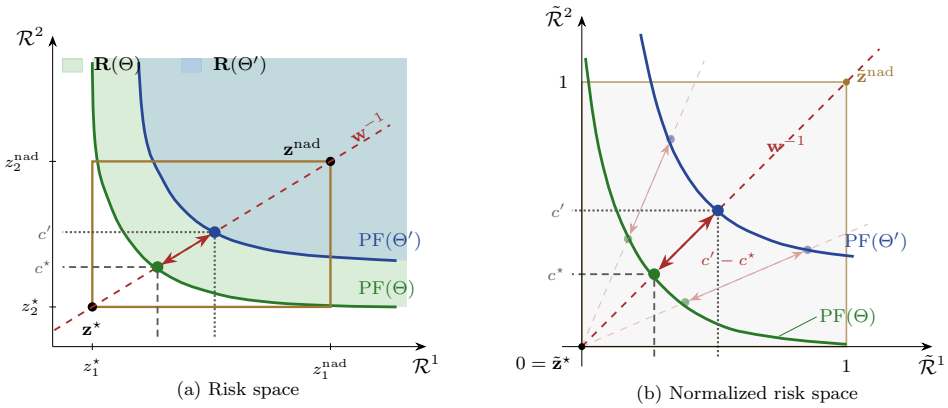

Figure 2.1: Conceptual illustration for $T = 2$ tasks. (a) The attainable risk regions $\mathbf{R}(\Theta) \supseteq \mathbf{R}(\Theta')$ and their Pareto fronts. The solid rectangle marks $[\mathbf{z}^\star, \mathbf{z}^{\text{nad}}]$; differing per-task risk ranges cause the Chebyshev contours (gray L-shapes) to be asymmetric. The dashed contour at level $c^\star$ is optimal over $\text{PF}(\Theta)$; the dotted contour at c' is optimal over $\text{PF}(\Theta')$. (b) After normalization (2.1), the rectangle maps to $[0,1]^2$ and the contours become symmetric. The gap $c' - c^\star > 0$ quantifies the cost of restricting the search to $\Theta'$and varies with the preference, as the faded rays illustrate. In chapter 3, $\Theta' = \Theta_{\text{conv}}$.

paradigm operates directly on the multi-objective gradient structure without reducing to a scalar objective. We review this paradigm, culminating in the EPO Search algorithm [40], which we use as a second training-based reference method.

Multiple Gradient Descent. The foundation is the Multiple Gradient Descent Algorithm (MGDA) of [41], which at each iteration finds a common descent direction that does not increase the risk of any objective. Let $G = [\nabla_\theta \mathcal{R}^1(\theta), \ldots, \nabla_\theta \mathcal{R}^T(\theta)] \in \mathbb{R}^{d \times T}$ denote the matrix of per-task gradients at θ, and let $\text{CH}_\theta = \{G\boldsymbol{\beta} : \boldsymbol{\beta} \in \overline{\Delta}^{T-1}\}$ denote the convex hull of these gradients. MGDA selects the direction $\mathbf{d}^\star = \text{argmin}_{\mathbf{d} \in \text{CH}_\theta} \|\mathbf{d}\|^2$, i.e., the minimum-norm element of the convex hull. A step $\theta \leftarrow \theta - \eta \mathbf{d}^\star$ decreases (or does not increase) every task loss simultaneously. If $\mathbf{d}^\star = \mathbf{0}$, the origin lies in the convex hull of the gradients, which is the first-order necessary condition for local Pareto optimality (*Pareto stationarity*). [17] made MGDA practical for deep networks by reformulating the minimum-norm problem in the dual, reducing the per-iteration cost from $O(d \cdot T^2)$ to $O(T^2)$ after a single pass through the gradients.

However, MGDA finds *some* Pareto-stationary point depending on the initialization, with no mechanism to steer toward a user-specified trade-off. [42] partially address this by decomposing the objective space into sub-regions, but their method does not guarantee convergence to the exact preference-specified trade-off and scales poorly with the number of tasks.

EPO Search. Exact Pareto Optimal (EPO) Search [40] resolves this limitation. Given a preference vector $\mathbf{w} \in \Delta^{T-1}$,[3] an **EPO solution** is a locally Pareto-optimal parameter $\theta^\star_{\mathbf{w}}$ satisfying

$$w_1 \mathcal{R}^1(\theta^\star_{\mathbf{w}}) \;=\; \cdots \;=\; w_T \mathcal{R}^T(\theta^\star_{\mathbf{w}}), \tag{2.2}$$

i.e., the preference-weighted losses are equalized. Geometrically, $\mathbf{R}(\theta^\star_{\mathbf{w}})$ lies at the intersection of the Pareto front with the ray through the origin in the direction $\mathbf{w}^{-1} = (1/w_1, \ldots, 1/w_T)$.

The key idea is to combine *descent* (toward the Pareto front) with carefully *controlled ascent* (along the front toward the desired preference). The algorithm maintains a non-uniformity measure quantifying the deviation of the current losses from the EPO condition (2.2); this measure is zero if and only if the condition holds. From the non-uniformity, the algorithm derives per-task adjustment weights $a_t \in \mathbb{R}$ that are positive for tasks whose preference-weighted loss exceeds the balanced level and negative for tasks below it. These adjustments define a **balancing direction** $\mathbf{d}_{\text{bal}} = G\mathbf{a}$: moving against $\mathbf{d}_{\text{bal}}$ decreases the losses of over-represented tasks while allowing the losses of under-represented tasks to increase, thereby improving the balance.

At each iteration, the algorithm solves a T-dimensional linear program (LP) to find a direction $\mathbf{d}_{\text{nd}} = G\boldsymbol{\beta}^\star$, $\beta^\star \in \overline{\Delta}^{T-1}$ in the convex hull of the gradients that balances two goals: reducing non-uniformity (when off the $\mathbf{w}^{-1}$ ray) and descending all objectives (when already on the ray, as in MGDA). The ascent is controlled by constraining the LP to always descend the objective with the currently highest preference-weighted loss, preventing divergence. Further details on the LP formulation and the adjustment weights are given in [40].

Convergence and normalization. Under mild regularity conditions and assuming only differentiability of the task losses, [40] prove that EPO Search converges to a locally Pareto-optimal solution satisfying (2.2), or, if no such solution exists, to the locally Pareto-optimal point closest to satisfying it. As with all gradient-based methods, these guarantees are local: the algorithm may reach different solutions depending on the initialization.

Unlike the augmented Chebyshev scalarization, EPO Search does not use utopia and nadir point normalization. Instead, it handles scale differences through an internal normalization of the preference-weighted losses. In our experiments, we apply EPO Search to the normalized risks $\tilde{\mathcal{R}}^t$ from section 2.1, which brings the preference semantics in line with the normalized

[3] The original paper uses the notation $\mathbf{r}$ for the preference vector. We use $\mathbf{w}$ for consistency with the scalarization methods of subsection 2.2.1. The semantics are aligned: in both cases, a larger w_t indicates greater importance of task t.

Chebyshev scalarization: under normalized risks, $\mathbf{w} = (1/T, \ldots, 1/T)$ corresponds to equal importance regardless of the raw loss scales. The convergence guarantees extend directly, since the normalized risks satisfy the required differentiability and non-negativity assumptions.

Role in this work. We use EPO Search alongside the augmented Chebyshev scalarization as a training-based reference method optimizing over the full parameter space Θ. The two methods complement each other: the Chebyshev scalarization reduces the problem to a single scalar objective, while EPO Search operates directly on the multi-gradient structure. Using two structurally different methods guards against attributing any observed gap between $\mathrm{PF}(\Theta_{\mathrm{conv}})$ and $\mathrm{PF}(\Theta)$ to artifacts of a particular optimization strategy.

2.3 Merging Methods

In this section, we introduce two data-free, optimization-free model merging methods that act at inference time. Both are instances of the task-arithmetic paradigm introduced by [11]. After presenting the methods as originally formulated, we investigate whether they can be used as steerable methods in the MOO sense of section 2.2, identify the restricted model class they search over, and state the central research question of this work.

Setting

Recall the pretrain-then-finetune paradigm that motivated model merging. We assume access to a shared pretrained model with parameters $\theta_{\mathrm{pre}} \in \Theta$ and, for each task $t \in [T]$, a fine-tuned model $f^t(\hat{\theta}^t, \cdot)$ obtained by optimizing $\hat{\mathcal{R}}^t$ starting from θ_{pre} with a standard iterative optimizer (e.g., SGD, Adam). The **task vector** of task t is defined as

$$\tau^t := \hat{\theta}^t - \theta_{\mathrm{pre}} \in \mathbb{R}^d.$$

2.3.1 Task Arithmetic

[11] introduced task vectors and showed that they can be manipulated through arithmetic operations to steer model behavior. The two principal operations are *addition*, which adds task vectors to the pretrained model to obtain a multi-task model, and *negation*, which subtracts a task vector to reduce performance on the corresponding task (e.g., for unlearning or debiasing [43]). Further operations, including task analogies, are explored in the original work. For multi-task merging, the method constructs:

$$\theta_{\mathrm{TA}}(\lambda) = \theta_{\mathrm{pre}} + \lambda \sum_{t=1}^{T} \tau^t, \tag{2.3}$$

where $\lambda \in \mathbb{R}$ is a global scaling factor, typically selected by grid search over a validation set. Positive λ adds task knowledge, while negative λ removes it. The useful range of λ is not a priori bounded: [11] showed that both extrapolation (λ beyond the range that corresponds to convex combinations) and negation ($\lambda < 0$ for individual task vectors) can be beneficial. In the special case $\lambda = 1/T$, (2.3) reduces to uniform weight averaging, commonly known as *model soups* [27]. All task vectors contribute equally and the only free parameter is λ.

2.3.2 TIES-Merging

TIES-Merging [12] addresses two sources of parameter interference that degrade the performance of task arithmetic: redundant (small-magnitude) parameter updates and sign disagreements across task vectors. Before merging, it applies a joint preprocessing $\Psi\colon (\mathbb{R}^d)^T \to (\mathbb{R}^d)^T$ to the collection of task vectors in three steps:

1. **Trim:** for each task vector τ^t, set all entries below a magnitude threshold (the top-k% by absolute value are retained) to zero.
2. **Elect sign:** for each parameter index $j \in [d]$, determine the majority sign across all nonzero entries $\tau_j^1, \ldots, \tau_j^T$ (weighted by magnitude).
3. **Disjoint merge:** for each parameter index j, retain only the entries whose sign agrees with the elected sign; set the rest to zero.

Let $(\tilde{\tau}^1, \ldots, \tilde{\tau}^T) = \Psi(\tau^1, \ldots, \tau^T)$ denote the preprocessed task vectors. The merged model is then

$$\theta_{\text{TIES}}(\lambda) = \theta_{\text{pre}} + \lambda \sum_{t=1}^{T} \tilde{\tau}^t, \tag{2.4}$$

with a global scaling factor λ as in task arithmetic.

2.3.3 Weight Interpolation as a Steerable Method

The formulations above use a single shared scaling factor λ and equal task weights. A natural question is whether per-task coefficients can navigate the Pareto front, turning these methods into steerable methods in the sense of section 2.2.

This idea has precedent. [44] proposed *rewarded soups*, which interpolate models fine-tuned on diverse reward functions with user-specified coefficients, explicitly framing weight interpolation as a tool for navigating the Pareto front across multiple objectives.

Motivated by this, we replace the uniform weighting and global scaling factor with per-task coefficients $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_T) \in \overline{\Delta}^{T-1}$, where $\overline{\Delta}^{T-1} = \{\boldsymbol{\alpha} \in$

$\mathbb{R}^T_{\geq 0} : \sum_{t=1}^T \alpha_t = 1\}$ denotes the closed simplex:

$$\theta_{\text{merge}}(\boldsymbol{\alpha}) = \theta_{\text{pre}} + \sum_{t=1}^{T} \alpha_t \, \tilde{\tau}^t, \tag{2.5}$$

where $\tilde{\tau}^t = \tau^t$ for task arithmetic and $(\tilde{\tau}^1, \ldots, \tilde{\tau}^T) = \Psi(\tau^1, \ldots, \tau^T)$ for TIES-Merging. Recall that the original formulations allows $\lambda \in \mathbb{R}$, so that the useful region of parameter space extends well beyond the simplex. Restricting to $\overline{\Delta}^{T-1}$ is a deliberate choice that defines a specific model class whose limitations we aim to quantify. The effective parameter set searched by these methods is

$$\Theta_{\text{conv}} = \left\{\theta_{\text{pre}} + \textstyle\sum_{t=1}^T \alpha_t \, \tilde{\tau}^t : \boldsymbol{\alpha} \in \overline{\Delta}^{T-1}\right\} = \theta_{\text{pre}} + \text{conv}\big(\{\tilde{\tau}^1, \ldots, \tilde{\tau}^T\}\big),$$

an at most $(T-1)$-dimensional convex subset of Θ.

The coefficient $\boldsymbol{\alpha}$ plays an analogous role to the preference vector $\mathbf{w}$ in the scalarization methods of section 2.2: varying $\boldsymbol{\alpha}$ over $\overline{\Delta}^{T-1}$ traces the image Θ_{conv} in parameter space, and the induced curve in objective space $\mathbb{R}^T_{\geq 0}$ is interpreted as an approximation to the Pareto front of Θ_{conv}. However, unlike the scalarization setting, there is no theoretical guarantee relating $\boldsymbol{\alpha}$ to a specific Pareto-optimal trade-off. The implicit assumption is that each per-task risk $\mathcal{R}^t(\theta_{\text{merge}}(\boldsymbol{\alpha}))$ is approximately linear in $\boldsymbol{\alpha}$, so that varying $\boldsymbol{\alpha}$ produces approximately proportional changes in objective space. The geometric conditions under which this assumption is warranted, and the circumstances under which it breaks down, are discussed in section 2.4.

Research Question

The methods introduced above, and more broadly all merging methods that preprocess task vectors and then search over their convex combinations [11, 12, 27, 44–47], or that determine coefficients at a more fine-grained level, such as per-layer or per-parameter [10, 14], are constrained to Θ_{conv} or low-dimensional generalizations thereof (for comprehensive surveys, see [9, 21, 48]). The training-based methods of section 2.2 and in particular EPO search (subsection 2.2.2), in contrast, optimize over the full parameter space Θ via gradient descent on the scalarized objective.

It is important to note that our setting assumes the fine-tuned models were obtained independently, without any modification of the fine-tuning procedure to facilitate merging. This excludes methods that intervene at post-training time, such as orthogonal fine-tuning [22, 48] or conditional language policy [13], which constrain the fine-tuning process to produce more mergeable models.

Given this setting, we arrive at the central question of this work:

Does the restriction to Θ_{conv} incur a significant approximation error relative to Θ? That is, does the Pareto front of Θ_{conv} contain trade-offs comparable to those achievable by training-based methods, or do there exist substantially better trade-offs outside the convex hull of (preprocessed) task vectors?

If the Pareto front of Θ_{conv} is strictly dominated by $\text{PF}(\Theta)$, this would indicate that the convex model class is fundamentally limited: no preprocessing heuristic applied to task vectors, regardless of its sophistication, can compensate for the low dimensionality of the search space.

2.4 Loss Landscape Geometry

The merging methods of section 2.3 rest on the assumption that each per-task risk $\mathcal{R}^t(\theta_{\text{merge}}(\boldsymbol{\alpha}))$ is approximately linear in the merging coefficient $\boldsymbol{\alpha}$. This requires, in particular, that the path through parameter space induced by varying $\boldsymbol{\alpha}$ does not pass through regions of high loss, since a loss barrier anywhere along this path would cause large and unpredictable changes in performance for small changes in $\boldsymbol{\alpha}$, violating the linearity assumption. We review two bodies of empirical work relevant to this question. The first concerns *mode connectivity*, the existence and nature of low-loss paths between independently trained models. The second concerns the *intrinsic dimension* of fine-tuning objectives, which characterizes the subspace structure of individual task solutions. Taken together, these results suggest that the linearity assumption is plausible under favorable conditions, but that these conditions are not guaranteed to hold in the setting of this thesis, and that Θ_{conv} may incur a non-trivial approximation error when they fail.

Mode Connectivity

Given two parameter vectors $\theta, \theta' \in \Theta$, a *loss barrier* on the linear interpolation between them refers to the phenomenon whereby the loss at some interior point $(1-\alpha)\theta + \alpha\theta'$, $\alpha \in (0,1)$, rises substantially above the loss at either endpoint. A naive view of the loss landscape imagines the optima found by independent training runs as isolated basins, with loss barriers separating any two distinct optima. [49] challenged this picture by showing that independently trained networks of the same architecture, obtained from different random initializations, are connected by simple curves of nearly constant low loss, even when the linear path between them passes through a high-loss barrier. They refer to this as *mode connectivity*. Concurrently, [50] reported similar findings, suggesting that mode connectivity is a general property of overparameterized networks. Crucially, the connecting paths in both works are not straight lines but piecewise-linear or Bézier curves whose interior points are found by an auxiliary optimization procedure. Mode con-

nectivity therefore does not imply that the linear interpolation between two solutions is free of loss barriers.

The conditions under which *linear* connectivity specifically holds were studied by [51]. Their setup (named *instability analysis*) trains two copies of a network from the same initialization but with different random orderings of the training data, inducing different stochastic gradient noise throughout training, and asks whether the loss along the straight-line interpolation between the two resulting solutions remains close to the loss at either endpoint. They find that standard vision models are initially unstable in this sense, but become stable early in training: once stability is reached, independently continued training runs converge to solutions with no high-loss barrier on the straight-line path between them. They attribute this to the shared initialization, which keeps both training trajectories within the same basin of the loss landscape. The pretrain-then-finetune paradigm is structurally similar, in that all expert models $\hat{\theta}^1, \ldots, \hat{\theta}^T$ share the common initialization θ_{pre}, suggesting that linear connectivity may hold between fine-tuned solutions. The results of [51], however, concern models trained on the *same* task under different stochastic noise; whether linear connectivity is preserved when models are fine-tuned on *different* tasks is a separate question.

[52] studied this question directly. Let $h^{(\ell)}(\theta, x) \in \mathbb{R}^{d_\ell}$ denote the vector of activations produced by layer ℓ of the network with parameters θ on input x, so that the final layer recovers the model's prediction $f^t(\theta, x)$. [52] introduce *Cross-Task Linearity* (CTL), which holds when, for $\alpha \in [0, 1]$ and any two tasks $s, t \in [T]$, the activations of the interpolated model with parameters $(1-\alpha)\hat{\theta}^s + \alpha\hat{\theta}^t$ satisfy

$$h^{(\ell)}\big((1-\alpha)\hat{\theta}^s + \alpha\hat{\theta}^t,\, x\big) \;\approx\; (1-\alpha)\, h^{(\ell)}(\hat{\theta}^s, x) \;+\; \alpha\, h^{(\ell)}(\hat{\theta}^t, x)$$

at each layer ℓ, for inputs x drawn from either task's domain. CTL is a strictly stronger property than mode connectivity: it requires not only that the loss remains low along the interpolation path, but that the model's internal representations change in a layerwise-linear fashion as the parameters are interpolated, which directly implies that the per-task risks are approximately linear in α. [52] provide empirical evidence that CTL consistently holds for models fine-tuned from a common pretrained checkpoint on different tasks, and conjecture that the pretrain-then-finetune paradigm causes networks to behave approximately as linear maps from parameter space to the space of layerwise activations. The bulk of the evidence, however, is from vision and vision-language models; the extent to which CTL holds for encoder-based language models of the kind used in this thesis is not established.

[23] offer a complementary account of when the linearity assumption is warranted. They identify *weight disentanglement* as a key enabling property, de-

fined informally as follows: the task vector τ^t primarily affects the model's predictions on inputs from $\mathcal{X}^t$, while having negligible effect on predictions for inputs from $\mathcal{X}^s$, $s \neq t$. When weight disentanglement holds, adding multiple task vectors has approximately additive effects on each task's performance, since each vector acts on a different part of the input domain, which supports the linearity of $\mathcal{R}^t(\theta_{\text{merge}}(\boldsymbol{\alpha}))$ in $\boldsymbol{\alpha}$. [23] demonstrate that weight disentanglement is an emergent property of pretraining, being largely absent at random initialization, and that it is amplified when models are fine-tuned in their tangent space, that is, when the fine-tuning update is constrained to be a first-order Taylor expansion of the model around θ_{pre}. The degree of disentanglement under standard, unconstrained fine-tuning is therefore lower than their best reported results, since tangent-space fine-tuning modifies the fine-tuning procedure and falls outside the scope of our setting. Moreover, their experiments are conducted exclusively on vision-language models, and the extent to which weight disentanglement emerges in encoder-based language models under standard fine-tuning is not established.

The low-loss structure between fine-tuned solutions was further characterized by [53]. They show that, given several independently trained models, one can find additional interior parameter vectors such that the resulting simplex lies approximately in a region of low loss. However, as with mode connectivity curves, finding these low-loss simplices requires an auxiliary optimization procedure of several training epochs. Θ_{conv} is not the result of such an optimization: it is the simplex whose vertices are the raw task vectors $\tilde{\tau}^1, \ldots, \tilde{\tau}^T$ translated by θ_{pre}, with no further adjustment of interior structure. The results of [53] therefore suggest that a favorable simplicial geometry may exist near Θ_{conv}, without implying that Θ_{conv} itself lies in a low-loss region.

In summary, the results reviewed above suggest that the linearity assumption is plausible when the merged models share a pretrained initialization and the tasks are not too dissimilar. The empirical basis for these phenomena rests predominantly on vision and vision-language architectures, and most results concern same-task or closely related models. Whether mode connectivity, cross-task linearity, and weight disentanglement extend to encoder-based language models fine-tuned on diverse NLP tasks remains an open question. Furthermore, even when the linearity assumption holds approximately, Θ_{conv} constitutes only a $(T-1)$-dimensional subset of Θ, and whether this restriction limits the achievable trade-offs is not settled by geometric considerations alone.

Intrinsic Task Dimension

A separate perspective on the structure of fine-tuning solutions is provided by work on the *intrinsic dimension* of optimization objectives. Following [54],

the intrinsic dimension d' of an optimization problem is the smallest integer such that optimizing within a randomly oriented d'-dimensional affine subspace of $\mathbb{R}^d$ suffices to reach a solution within a fixed tolerance of the full-dimensional optimum. [55] apply this concept to language model fine-tuning and find that pretrained language models have surprisingly low intrinsic dimension across downstream tasks: for instance, fine-tuning RoBERTa [56] on MRPC [57] (a model and dataset also used in our experiment) to within 90% of full-parameter performance requires optimizing only approximately 200 parameters in a randomly projected subspace of $\mathbb{R}^d$. They further show that pretraining implicitly minimizes intrinsic dimension, and that the intrinsic dimension varies substantially across tasks, with tasks such as MRPC admitting solutions in very low-dimensional subspaces while tasks such as MNLI require substantially higher-dimensional ones.

Two aspects of this result bear on the merging setting. First, since the low-dimensional subspaces in which fine-tuning succeeds are defined by random projection, many such subspaces exist for any given task rather than a single canonical one. Intrinsic dimension results therefore do not identify a unique subspace in which each task's fine-tuning solutions reside, and one cannot directly infer from them that the task vectors of different tasks span suboptimal regions of parameter space. Second, the substantial variation in intrinsic dimension across tasks implies that different tasks make qualitatively different demands on the parameter space. Whether Θ_{conv} adequately captures trade-offs between tasks with differing intrinsic dimensions is not determined by these results alone, but further motivates the experimental comparison of chapter 3.

Chapter 3

Experiments

Motivated by the exposition in chapter 2, we now design an experiment to quantify the approximation error incurred by restricting the parameter search to Θ_{conv}. Before presenting the experimental setup, we situate our measurement within the hierarchy of approximation errors that accumulate along the modeling pipeline.

Hierarchy of approximation errors. The gap between the Bayes-optimal risk on a task t and the risk of any concrete merged model can be decomposed into a chain of non-negative terms, each attributable to a specific modeling decision:

$$\underbrace{\mathcal{E}^t(\hat{\theta})}_{\text{total excess risk}} = \underbrace{\mathcal{E}^t_{\text{appr}}(\Theta)}_{\substack{\text{architecture}\\ \text{(RoBERTa)}}} + \underbrace{\left[\inf_{\theta\in\Theta'} \mathcal{R}^t(\theta) - \inf_{\theta\in\Theta} \mathcal{R}^t(\theta)\right]}_{\substack{\text{restriction to}\\ \text{model class } \Theta'}} + \underbrace{\left[\mathcal{R}^t(\hat{\theta}) - \inf_{\theta\in\Theta'} \mathcal{R}^t(\theta)\right]}_{\substack{\text{optimization +}\\ \text{estimation error}}}, \tag{3.1}$$

where $\Theta' \subseteq \Theta$ is the effective parameter set searched by the method (either Θ_{conv} for merging methods or Θ for training-based methods) and $\hat{\theta} \in \Theta'$ is the parameter vector returned by the method. The first term is the price of the architecture and is shared across all methods. The second term is zero for training-based methods that search over Θ and positive for merging methods constrained to $\Theta_{\text{conv}} \subset \Theta$; this is the quantity our experiment targets. The third term conflates finite-sample effects and optimization suboptimality (convergence to local rather than global optima); it is present for all methods but may differ in magnitude between merging and training-based approaches.

Crucially, the restriction term can only be isolated if the optimization and estimation errors of the training-based methods are small relative to the gap we wish to detect. If the training-based methods themselves underperform due to suboptimal hyperparameters or local minima, we obtain only a *lower*

bound on the true restriction error. We address this concern with a calibration experiment in subsection 3.2.1.

3.1 Methodology

3.1.1 Model and Head Architecture

We use RoBERTa-base [56] as the shared pretrained model θ_{pre}. RoBERTa is an encoder-only transformer with approximately 125 million parameters, pretrained on a large English corpus with a masked language modeling objective. For each task $t \in [T]$, the model is equipped with a task-specific binary classification head consisting of two layers (768×768 and 768×2) mapping the encoder output to $\mathcal{Y}^t = \{0, 1\}$.

Following standard practice in multi-task learning with encoder models [16, 17], we maintain T separate classification heads throughout all experiments, while the encoder is shared across tasks (obtained either by merging or by joint training, as detailed below). This architectural choice is essential: each head is trained under a different loss function with task-specific label semantics (e.g., grammatical acceptability for CoLA vs. semantic equivalence for MRPC), and sharing or interpolating heads across tasks would conflate encoder-level interference with head incompatibility. Since the classification heads never shared a common pretraining, there is no reason to hope for the phenomena of section 2.4. The full parameter vector accordingly decomposes as $\theta = (\theta_{\text{enc}}, \theta^1_{\text{head}}, \ldots, \theta^T_{\text{head}})$, where θ_{enc} collects the shared encoder parameters and θ^t_{head} the head parameters for task t. At inference time on task t, the input passes through the encoder and is routed to head t; no other head is involved. This distinction is an artifact of choosing encoder only models like RoBERTa. Other commonly used model merging benchmark settings don't share this complication (section 3.3).

Implications for merging. In the model merging methods, the task vectors and the convex combination operate exclusively on the encoder parameters. Let $\tau^t_{\text{enc}} = \hat{\theta}^t_{\text{enc}} - \theta_{\text{pre,enc}}$ denote the encoder-level task vector for task t. The merged model evaluated on task t uses

$$\theta^t_{\text{merge}}(\boldsymbol{\alpha}) = \Big(\theta_{\text{pre,enc}} + \sum_{s=1}^{T} \alpha_s \, \tilde{\tau}^s_{\text{enc}}, \;\; \hat{\theta}^t_{\text{head}}\Big), \tag{3.2}$$

where $\tilde{\tau}^s_{\text{enc}} = \tau^s_{\text{enc}}$ for Task Arithmetic and $(\tilde{\tau}^1_{\text{enc}}, \ldots, \tilde{\tau}^T_{\text{enc}}) = \Psi(\tau^1_{\text{enc}}, \ldots, \tau^T_{\text{enc}})$ for TIES-Merging. Note that the encoder component is identical across tasks; the superscript t indicates that at inference time on task t, the shared merged encoder is paired with the head of task t. The convex set Θ_{conv} is thus defined over the encoder subspace:

$$\Theta_{\text{conv}} = \big\{\theta_{\text{pre,enc}} + \textstyle\sum_{t=1}^{T} \alpha_t \, \tilde{\tau}^t_{\text{enc}} : \boldsymbol{\alpha} \in \overline{\Delta}^{T-1}\big\},$$

an at most $(T-1)$-dimensional convex subset of the encoder parameter space. This is the set whose approximation error we aim to quantify.

Implications for training-based methods. For the Chebyshev scalarization and EPO Search, the model is initialized from θ_{pre} with T randomly initialized classification heads, and all parameters (encoder and heads) are optimized jointly. During each training step, a batch from task t passes through the shared encoder and through head t only; gradients from task t's loss update the encoder and head t, but not heads $s \neq t$.

This architectural decomposition ensures that both merging and training-based methods use T separate task-specific heads. However, the heads differ in origin: merging methods reuse the expert heads trained jointly with each expert's encoder, while the training-based methods learn new heads from random initialization alongside the shared encoder. A cleaner comparison would freeze the expert heads and use them for both merging and training-based methods, purely isolating encoder-level differences; we did not implement this and acknowledge the resulting confound in section 3.3.

3.1.2 Tasks and Datasets

We evaluate on a subset of the General Language Understanding Evaluation (GLUE) benchmark [57]. We restrict attention to binary classification tasks in order to maintain uniform head architecture across all tasks, avoiding the confound of merging classification heads with different output dimensions. This restriction is non-trivial: it excludes the largest and most challenging GLUE task (MNLI, 3 classes, 393k samples), which limits the generality of our findings (see section 3.3). The four selected tasks are:

CoLA (Corpus of Linguistic Acceptability, $N_{\text{train}} \approx 8.5$k): Classify whether a sentence is grammatically acceptable. The canonical evaluation metric is Matthews Correlation Coefficient (MCC).[1]

MRPC (Microsoft Research Paraphrase Corpus, $N_{\text{train}} \approx 3.7$k): Classify whether two sentences are semantically equivalent. Evaluated by accuracy.[2]

QNLI (Stanford Question Answering Dataset, $N_{\text{train}} \approx 105$k): Classify whether a context sentence contains the answer to a question. Evaluated by accuracy.

[1]The Matthews Correlation Coefficient is defined as $\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$, where TP, TN, FP, FN denote the entries of the confusion matrix. MCC ranges in $[-1, 1]$, with 0 corresponding to random prediction and 1 to perfect classification. It is preferred over accuracy for CoLA due to label imbalance.

[2]The canonical GLUE metric for MRPC is the average of accuracy and F1 score. We report accuracy for consistency across tasks since results are similar

SST-2 (Stanford Sentiment Treebank, $N_{\text{train}} \approx 67$k): Binary sentiment classification of movie reviews. Evaluated by accuracy.

Training sample sizes span two orders of magnitude, from 3.7k (MRPC) to 105k (QNLI), introducing a natural asymmetry in fine-tuning quality and task vector magnitude.

In addition to the four-task benchmark, we conduct a separate two-task experiment on the pair (MRPC, QNLI), chosen because preliminary cross-evaluation (see subsection 3.1.5) revealed substantial destructive interference between their encoder-level task vectors. This pair allows a focused study of the convex restriction under unfavorable conditions.

3.1.3 Data Splits

The GLUE authors keep the original test labels private. Following common practice, we repurpose the published validation split for evaluation. However, because some of the fine-tuned models we use were trained on the full original training split, and because the training-based methods require a validation set for early stopping and hyperparameter selection, we subdivide the original validation split into two equal halves: a *validation split* and a *test split*. We verified that this subdivision does not introduce label imbalance beyond that already present in the original split. Table 3.1 reports the resulting split sizes.

Table 3.1: Dataset split sizes after subdividing the original GLUE validation set. All reported experimental results use the test split.

Task	Train	Validation	Test
CoLA	8,551	521	522
MRPC	3,668	204	204
QNLI	104,743	2731	2732
SST-2	67,349	436	436

We note that the MRPC test split contains only 204 samples. For binary classification at an accuracy of approximately 85%, a normal approximation to the binomial gives a 95% confidence interval of roughly ± 5 percentage points. Results on MRPC should therefore be interpreted with appropriate caution, and we do not draw conclusions from differences smaller than this margin. For the empirical standard deviation of MRPC evaluated across four training seeds, see Appendix C.

3.1.4 Evaluation Metrics and Normalization

Per-task metrics. For CoLA we report MCC, as this is standard practice due to a 70% vs. 30% class imbalance; the remaining tasks are evaluated by classification accuracy. For all methods, we report test-split performance. The training-based methods optimize the empirical cross-entropy loss $\hat{\mathcal{R}}^t$ as their training objective; the evaluation metrics (MCC, accuracy) are computed from the same predictions but may differ from the training loss.

When reporting an aggregate metric across tasks with unequal evaluation set sizes, two conventions exist. *Micro-averaged* metrics pool all predictions across tasks into a single confusion matrix, so that tasks with larger evaluation sets contribute proportionally more to the aggregate; in our setting, QNLI and SST-2 would dominate. *Macro-averaged* metrics compute the metric independently for each task and then average the per-task values, weighting each task equally regardless of evaluation set size. We report macro-averaged accuracy where an aggregate is needed since our goal is to assess trade-offs across tasks of equal importance rather than to reflect the distribution of evaluation samples.

Utopia and nadir points. The normalized Chebyshev scalarization (2.1) and the normalized EPO Search objective require estimates of the utopia and nadir points to normalize the per-task losses so that the scalarization treats tasks fairly regardless of their absolute loss scales.

We approximate the utopia point $\mathbf{z}^\star$ componentwise by the cross-entropy loss of each task's individually fine-tuned expert $\hat{\theta}^t$ on task t, i.e., $z_t^\star \approx \hat{\mathcal{R}}^t(\hat{\theta}^t)$. This is an approximation to $z_t^\star(\Theta) = \inf_{\theta\in\Theta} \mathcal{R}^t(\theta)$; the fine-tuned expert may not be the global optimum over Θ, but it provides a consistent and reproducible reference.

For the nadir point, we use the componentwise worst cross-entropy loss among all fine-tuned experts evaluated across tasks with separate heads:

$$z_t^{\text{nad}} \approx \max_{s\in[T]} \hat{\mathcal{R}}^t(\hat{\theta}^s_{\text{enc}}, \hat{\theta}^t_{\text{head}}). \tag{3.3}$$

Each expert's encoder is paired with task t's own head, so that the nadir reflects encoder-level interference rather than head incompatibility (cf. subsection 3.1.5).

The utopia approximation is straightforward: single-task fine-tuning directly targets the per-task risk infimum, and the only gap is optimization suboptimality of the expert. The nadir approximation is a heuristic. The true nadir is defined as the componentwise worst risk over the entire Pareto set (Definition 2.4), which is unknown. We approximate it using cross-evaluation of single-task experts, on the assumption that these maximally specialized

models are approximately Pareto-optimal and cause the most severe cross-task interference. This estimate may be inaccurate in either direction: it may overestimate the nadir if the experts are not truly Pareto-optimal (e.g., due to positive transfer between tasks), or underestimate it if the true Pareto set contains solutions with worse cross-task interference than any single-task expert. The utopia and nadir points are computed on the validation split to avoid data leakage from the test split for training-based methods.

We note that the cross-evaluation nadir can exceed the per-task loss at initialization ($\approx \ln 2 \approx 0.693$ for binary tasks with a randomly initialized head), because an off-task expert's encoder may actively interfere with predictions even through the correct head. In such cases, the normalized risk at initialization is less than 1, and the training trajectory does not span the full interval $[0, 1]$. We accept this: the normalization should reflect the range of losses attainable by the Pareto set, not the range visited during a particular training run. A scalarization calibrated to the Pareto set ensures that the preference vector $\mathbf{w}$ has consistent semantics across tasks, even if the optimization starts from an interior point of the normalized space.

3.1.5 Expert Models and Cross-Evaluation

As fine-tuned expert models, we use the publicly available RoBERTa-base checkpoints published by TextAttack [58]. The hyperparameters used for fine-tuning these models are reported in Appendix B. We do not control the fine-tuning procedure of these experts; any suboptimality in the expert models propagates into the task vectors and thereby into the merging methods. This is an inherent limitation of our setup but reflects the realistic scenario motivating model merging, in which practitioners reuse existing artifacts without modifying the fine-tuning process.

To characterize encoder-level inter-task interference, we evaluate each expert's encoder $\hat{\theta}^s_{\text{enc}}$ paired with every task's head $\hat{\theta}^t_{\text{head}}$ on task t's original validation data, yielding a $T \times T$ cross-evaluation matrix C with entries $C_{st} = \mathcal{R}^t(\hat{\theta}^s_{\text{enc}}, \hat{\theta}^t_{\text{head}})$. The diagonal entries C_{tt} correspond to in-distribution performance and serve as the utopia point estimates. The off-diagonal entries C_{st} ($s \neq t$) measure how well an encoder specialized on task s supports predictions on task t through t's own head, isolating encoder-level transfer from head incompatibility.

High off-diagonal performance indicates that the encoder-level task vector τ^s_{enc} does not destructively interfere with task t's representations, suggesting that weight disentanglement (section 2.4) approximately holds for the pair (s, t). Conversely, off-diagonal performance near chance level indicates that the encoder trained on task s does not preserve useful features for task t, predicting that the convex combination Θ_{conv} may not contain favorable trade-offs for this pair.

Table 3.2: Cross-task zero-shot evaluation accuracy of fine-tuned model on the original validation split

Model ↓ / Task →	CoLA	MRPC	QNLI	SST-2
CoLA	**0.8504**	0.6765	0.4948	0.5195
MRPC	0.3384	**0.9118**	0.4435	0.4897
QNLI	0.6347	0.2917	**0.9268**	0.4839
SST-2	0.4775	0.4877	0.4825	**0.9404**

3.1.6 Training-Based Methods: Implementation

We implement the augmented normalized Chebyshev scalarization (2.1) and EPO Search (subsection 2.2.2) as training-based reference methods, both operating on the full parameter space Θ.

Non-differentiability of the Chebyshev objective. The max operator in the Chebyshev scalarization is non-differentiable at points where two or more preference-weighted losses coincide, i.e., where $w_s\tilde{\mathcal{R}}^s(\theta) = w_t\tilde{\mathcal{R}}^t(\theta)$ for some $s \neq t$. This is precisely the regime the scalarization is designed to reach: balanced trade-offs correspond to ties in the preference-weighted losses. In practice, automatic differentiation propagates gradients through the max via subgradient selection: the gradient flows through the single argmax task, and when multiple tasks tie, an arbitrary winner is selected. This can in principle cause oscillatory behavior near balanced trade-offs, since consecutive steps may descend different tasks. The augmentation term $\rho \sum_t w_t\tilde{\mathcal{R}}^t(\theta)$ mitigates this effect by contributing gradients from all tasks regardless of the argmax, providing a stabilizing force proportional to ρ. We did not observe training instabilities attributable to the non-differentiability in our experiments.

Sample balancing. The training sample sizes vary from 3.7k (MRPC) to 105k (QNLI). To prevent the training dynamics from being dominated by large-sample tasks, we cap the number of samples per task at 6,000 per epoch. For tasks exceeding this cap, we draw a random subset of 6,000 samples at each epoch, using a seed that incorporates the epoch number to ensure varying subsets across epochs. For tasks below the cap (MRPC), the full training set is used at every epoch, and the dataloader is restarted when exhausted. This strategy ensures roughly equal gradient contributions per task per epoch but introduces an asymmetry in cumulative data exposure: over E epochs, the model sees up to $\min(E \times 6{,}000,\ N^t_{\text{train}})$ unique samples from large tasks but the same $\sim$3,700 MRPC samples E times. Whether this asymmetry systematically advantages or disadvantages either task in the multi-task objective is not clear a priori and may depend on the preference vector.

The cap of 6,000 was chosen to keep training within computational budget. We did not conduct a sensitivity analysis over this value; however, early stopping consistently triggered before all epochs were exhausted, suggesting that the training-based methods were not data-starved at this cap. Since the cap only affects the training-based methods and not the merging methods (which use fixed expert models), a higher cap could only narrow or maintain the optimization error of the training-based methods, potentially widening the observed gap to merging.

Further training details. We use the AdamW optimizer [59]. Gradient checkpointing is enabled to reduce GPU memory consumption.[3] For the two-task benchmark (MRPC–QNLI), we run the Chebyshev scalarization with four random seeds to obtain variance estimates; all other configurations use a single seed. The full set of hyperparameters, including learning rate, schedule, number of epochs, and augmentation parameter ρ, is reported in Appendix B.

3.1.7 Merging Methods: Implementation

We implement Task Arithmetic and TIES-Merging as described in chapter 2, operating on encoder-level task vectors τ^t_{enc} with per-task merging coefficients $\boldsymbol{\alpha} \in \overline{\Delta}^{T-1}$ as in (3.2). For TIES-Merging, the trim, elect-sign, and disjoint-merge preprocessing Ψ is applied to the encoder-level task vectors only; head parameters are excluded. We follow the TIES authors and retain the top $k = 20\%$ of parameters by absolute value in the trim step. Since both methods are optimization-free, evaluation at a given $\boldsymbol{\alpha}$ requires only constructing the merged encoder, attaching the appropriate expert head, and performing a forward pass over the test data, with negligible computational cost compared to the training-based methods.

3.1.8 Preference Vectors

Four-task benchmark. We evaluate all methods at seven preference vectors, chosen to probe interpretable regions of the Pareto front rather than to densely cover the 3-simplex:

1. **Uniform:** $\mathbf{w} = (0.25, 0.25, 0.25, 0.25)$.
2. **Single-task dominant** ($\times 4$): $w_t = 0.7$ for the focal task and $w_s = 0.1$ for $s \neq t$.
3. **Balanced pairs** ($\times 2$): $(0.4, 0.1, 0.4, 0.1)$ emphasizing (CoLA, QNLI) and $(0.1, 0.4, 0.1, 0.4)$ emphasizing (MRPC, SST-2).

[3] Gradient checkpointing recomputes intermediate activations during the backward pass rather than storing them, trading computation for memory. This does not affect the computed gradients or the optimization trajectory.

This selection is deliberately sparse; a uniform grid at resolution 0.1 on Δ^3 would yield $\binom{13}{3} = 286$ points, which is computationally feasible for merging methods but expensive for training-based methods. We note that the merging methods could be evaluated at arbitrary density at negligible cost, but we restrict to the same preference vectors for comparability.

Additionally, we evaluate the training-based methods at unit preference vectors e_t for the calibration experiment of subsection 3.2.1.

Two-task benchmark (MRPC–QNLI). We evaluate at nine uniformly spaced preference vectors on Δ^1:

$$\mathbf{w} \in \{(0.9, 0.1),\ (0.8, 0.2),\ \ldots,\ (0.1, 0.9)\},$$

providing a dense sweep of the one-dimensional Pareto front.

For the merging methods, the correspondence between the merging coefficient $\boldsymbol{\alpha}$ and the preference vector $\mathbf{w}$ is direct: we set $\boldsymbol{\alpha} = \mathbf{w}$. As discussed in section 2.3, this identification assumes approximate linearity of the per-task risks in $\boldsymbol{\alpha}$, an assumption that may fail under destructive interference.

3.2 Results

3.2.1 Calibration: Single-Task Recovery

Before comparing merging and training-based methods on the multi-task objective, we verify that the training pipeline does not introduce a systematic optimization deficit relative to the TextAttack experts. To this end, we train the Chebyshev and EPO methods with extreme preference vectors that place all weight on a single task and compare the resulting single task performance to the corresponding TextAttack expert $\hat{\theta}^t$. The hyperparameters are reported in Appendix B.

Table 3.3: Single-task calibration. Performance of training-based methods at extreme preference vectors compared to the TextAttack fine-tuned experts. Metric: MCC for CoLA, accuracy (%) for others.

Method	CoLA (MCC)	MRPC (Acc)	QNLI (Acc)	SST-2 (Acc)
TextAttack expert	0.638	91.2%	92.3%	94.0%
Chebyshev ($\mathbf{w} = \mathbf{e}_t$)	0.566	88.2%	92.1%	94.3%
EPO ($\mathbf{w} = \mathbf{e}_t$)	0.565	91.4%	91.9%	94.5%

For MRPC, QNLI, and SST-2, Chebyshev scalarization and EPO search are competitive with the TextAttack experts. Since we only ran the calibration experiment with a single seed, the 2% gap of Chebyshev scalarization on

MRPC could be attributed to noise and an insufficient evaluation set size (see subsection 3.1.3). On CoLa, our training pipeline and methods exhibit a substantial performance gap. This under-performance could lead to merging methods being more competitive than they should be with our training based methods, which could result in a more conservative estimate of the approximation error. The gap could be attributed to the lack of early stopping and subsequent overfitting to the training data in our calibration experiment.

3.2.2 Four-Task Benchmark

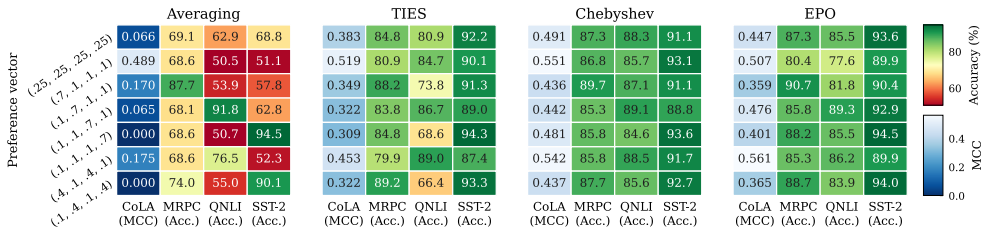

Figure 3.1: Four-task benchmark results. Each panel shows one method; rows correspond to preference vectors $\mathbf{w}$ and columns to per-task performance (MCC for CoLA, accuracy for others). Color encodes accuracy on a shared percentage scale (right upper axis); CoLA uses a separate MCC color scale (right lower axis)

The most striking pattern in Figure 3.1 is the disparity between merging and training-based methods on CoLA. Averaging drops below 0.10 MCC on four of seven preference vectors and reaches exactly 0.000 at both the SST-2-dominant and MRPC+SST-2 pair preferences. We were unable to identify an error in the evaluation pipeline; the merged encoder at these operating points appears to have lost all information relevant to grammatical acceptability. TIES-Merging recovers substantial CoLA performance (MCC $\geq$ 0.30 across all preferences), confirming that the trim-and-elect heuristic mitigates destructive interference, but a persistent gap to the training-based methods remains: both Chebyshev and EPO maintain MCC above 0.35 at every preference vector, including those that strongly de-emphasize CoLA.

A second pattern concerns the stability of non-focal tasks under shifting preferences. Averaging exhibits catastrophic degradation: preferences that emphasize a single task routinely cause unrelated tasks to fall to or near chance level. TIES-Merging reduces the severity of this effect but does not eliminate it, with QNLI dropping below 70% at two preference vectors. The training-based methods, in contrast, degrade gracefully: under Chebyshev, no task falls below 84.6% accuracy or 0.436 MCC at any tested preference. This suggests that gradient-based optimization reaches regions of parameter space where task performance is jointly high and less sensitive to the trade-off direction, consistent with the hypothesis that Θ contains solutions with

lower inter-task interference than those accessible in Θ_{conv}.

The gap between merging and training-based methods is not uniform across tasks. On SST-2, all methods except averaging perform comparably well, with even averaging reaching 94.5% at its best operating point. This robustness suggests that sentiment classification relies on encoder features that survive interpolation with other task vectors. The gap is largest on CoLA and QNLI, where the merging methods show severe degradation under unfavorable preferences. This asymmetry is consistent with the intuition that CoLA requires fine-grained syntactic representations that are easily disrupted by task vectors trained on semantic tasks, while SST-2's signal is more robust to encoder perturbation.

Notably, the training-based methods exhibit comparatively mild trade-offs between tasks: while some variation in non-focal task performance is visible across preference vectors, it is far less severe than the degradation observed under averaging or TIES-Merging. As discussed in Remark 2.3, this is expected in an MTL setting with sufficient model capacity, where the Pareto front can in principle collapse to a single point and any observed trade-off is attributable to optimization limitations or restricted model classes rather than fundamental objective conflict. The relatively mild trade-offs for the training-based methods are consistent with RoBERTa-base having sufficient capacity to accommodate all four binary classification tasks simultaneously, though we cannot rule out that a more challenging task combination would reveal sharper trade-offs even for training-based methods. Whether a more pronounced gap between $\text{PF}(\Theta_{\text{conv}})$ and $\text{PF}(\Theta)$ would emerge under genuinely conflicting objectives, for instance in a multi-objective learning setting with competing criteria such as helpfulness and harmlessness, remains an open question.

Comparing the two training-based methods, Chebyshev and EPO produce broadly similar performance but differ in their response to extreme preferences. EPO achieves the highest CoLA MCC of any method (0.561 at the CoLA+QNLI pair preference) but shows larger fluctuations on non-focal tasks, while Chebyshev maintains more uniform performance across all tasks at every preference vector. This difference is consistent with their respective designs: the Chebyshev scalarization penalizes the worst-case weighted loss and therefore resists sacrificing any single task, while EPO's balancing mechanism allows controlled ascent on non-focal tasks to achieve tighter preference alignment.

Among the merging methods, the improvement from averaging to TIES-Merging is substantial and consistent. At the uniform preference, TIES improves over averaging by $+0.317$ MCC on CoLA, $+15.7$ percentage points on MRPC, $+18.0$ on QNLI, and $+23.4$ on SST-2. However, this improvement narrows the gap to training-based methods without closing it: at the

uniform preference, Chebyshev exceeds TIES on CoLA, MRPC, and QNLI, with only SST-2 slightly favoring TIES. The pattern persists across preference vectors. TIES approaches training-based performance on SST-2 and MRPC but retains a clear deficit on CoLA and QNLI, where destructive interference in Θ_{conv} is most pronounced.

Since the training-based methods find only locally Pareto-optimal solutions, the gaps reported here are lower bounds on the true restriction error $\text{PF}(\Theta_{\text{conv}})$ vs. $\text{PF}(\Theta)$; on CoLA, where the calibration experiment revealed a non-trivial optimization deficit, this lower bound may not be tight.

3.2.3 Two-Task Benchmark: MRPC–QNLI

The two-task benchmark allows a dense sweep of the Pareto front, providing a more detailed picture of where Θ_{conv} falls short. Figure 3.2 plots the Pareto front in metric space; full numerical results with standard deviations are reported in Table C.1.

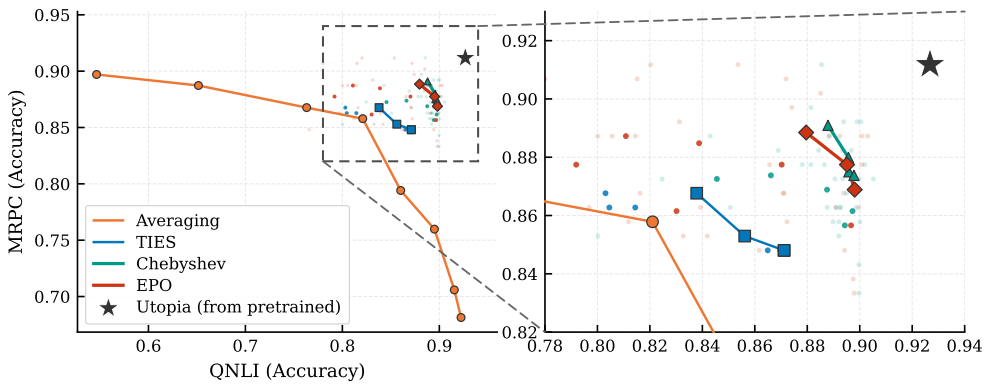

Figure 3.2: Two-task benchmark (MRPC–QNLI). Each opaque point represents the mean performance for a single preference vector across four random seeds; individual seed runs are shown as semi-transparent scatter points. Mean points belonging to the Pareto front are connected per method. The star marks the utopia point computed from TextAttack expert models on the test split, so that the reference point and the plotted results are computed on the same data; the utopia point used for loss normalization during training is computed on the validation split to avoid data leakage (section 3.1.4).

The left panel of Figure 3.2 immediately reveals the central finding: averaging traces a wide arc through objective space, reaching QNLI accuracy as low as 54.6% (near chance) at the most MRPC-dominant preference, while the training-based methods cluster in a compact region near the utopia point. As discussed in Remark 2.3, the expected behavior in an MTL setting with sufficient model capacity is collapse of the Pareto front toward the utopia point, since the tasks do not conflict by construction. The large arc traced by averaging therefore does not reflect a fundamental trade-off between MRPC

and QNLI; it reflects the inability of Θ_{conv} to represent a model that performs well on both tasks simultaneously at intermediate merging coefficients.

The wide arc traced by averaging reflects severe destructive interference between the two encoder-level task vectors: at intermediate merging coefficients, the merged encoder fails to preserve the representations needed by either task, producing performance substantially below what either expert achieves individually. A detailed comparison against the predictions of the linearity assumption (section 2.4) would require analysis in risk space rather than metric space and is left to future work.

At the most QNLI-dominant preferences, averaging achieves the highest raw QNLI accuracy of any method (92.2%, compared to ∼89.6% for the training-based methods). This occurs because the merging coefficient $\boldsymbol{\alpha} = (0.1, 0.9)$ places the merged encoder close to the QNLI expert, largely recovering its single-task performance. The training-based methods do not reach this level because the Chebyshev scalarization resists the corresponding collapse of MRPC: even at $\mathbf{w} = (0.1, 0.9)$, the MRPC task still contributes to the objective through both the augmentation term and its $w_{\text{MRPC}} = 0.1$ weight in the max term, preventing full sacrifice of MRPC performance. This illustrates a structural difference between the two approaches: merging methods can reach the individual experts by construction (at the simplex vertices), while training-based scalarization methods trade a small deficit on the dominant task for substantially better performance on the non-dominant one.

TIES-Merging substantially compresses the Pareto front relative to averaging, keeping QNLI accuracy above 80% at all preferences. However, the zoomed right panel of Figure 3.2 reveals a persistent and clearly visible gap between the TIES front and the training-based fronts. The TIES markers consistently occupy the lower-left region of the zoomed panel, achieving lower accuracy on both tasks simultaneously at most operating points. This gap is the direct empirical evidence for our central claim: the convex set Θ_{conv}, even after TIES preprocessing, does not contain trade-offs comparable to those achievable by unconstrained optimization over Θ. No further sophistication in the choice of merging coefficients $\boldsymbol{\alpha}$ can close this gap, since the merging methods already search in a dense grid over the simplex; the limitation is in the geometry of the set itself.

TIES also exhibits a saturation effect at QNLI-dominant preferences: from $\mathbf{w} = (0.3, 0.7)$ through $(0.1, 0.9)$, both QNLI and MRPC accuracy plateau at approximately 87% and 84.8% respectively. Varying the merging coefficient beyond this point produces no further movement along the Pareto front. This saturation likely reflects the sparsity introduced by the TIES preprocessing: after trimming 80% of entries and resolving sign conflicts, the preprocessed task vectors have limited overlap, and beyond a certain $\boldsymbol{\alpha}$ the contribution of the minority task vector becomes too small to influence

predictions.

Turning to the training-based methods, Chebyshev and EPO trace broadly overlapping Pareto fronts, though they are not identical. Chebyshev maintains both tasks above 85% accuracy across the full preference sweep, while EPO drops to 79.6% on QNLI at the most MRPC-dominant preference and achieves somewhat higher MRPC accuracy at balanced and MRPC-dominant operating points. The zoomed panel shows that the two methods occupy a similar region of objective space, with Chebyshev producing a slightly tighter cluster and EPO exhibiting more spread. In this two-task setting, the additional machinery of EPO Search does not appear to confer a clear advantage over the simpler Chebyshev scalarization, though this conclusion may not extend to settings with more tasks or more complex trade-offs.

The variance across random seeds merits discussion. For Chebyshev, standard deviations range from 0.3 to 2.4 percentage points on QNLI and 0.0 to 1.9 on MRPC. EPO shows notably higher variance, reaching ± 2.7 on QNLI and ± 3.2 on MRPC, visible in the scatter of semi-transparent points in the zoomed panel. With four seeds, these variance estimates are themselves uncertain, and we cannot conclusively determine whether the higher EPO variance reflects genuine sensitivity of the EPO algorithm to initialization or suboptimal hyperparameter tuning for specific preference vectors. However, even accounting for this variance, the gap between the training-based methods and TIES remains larger than the seed-to-seed variation at most operating points, supporting the conclusion that the observed gap reflects a structural limitation of Θ_{conv} rather than an artifact of particular random seeds.

As with the four-task benchmark, the training-based front can only overestimate the true $\text{PF}(\Theta)$, so the true restriction error may be larger than what is visible in Figure 3.2.

3.3 Limitations

We discuss the principal limitations of our experimental design, organized by the terms in the error decomposition (3.1).

Single architecture and modality. All experiments use RoBERTa-base, an encoder-only transformer with 125M parameters, evaluated on English natural language understanding tasks. The mode connectivity and cross-task linearity results reviewed in section 2.4 were established predominantly on vision and vision-language models (e.g., CLIP, ViT). Whether our findings extend to decoder-only language models, larger parameter scales, or non-English tasks remains an open question. Furthermore, the encoder only nature of RoBERTa forces us to use multiple classification heads, which is

common in older MTL literature but not relevant in the previously mentioned models or LLMs.

Head asymmetry. The merging methods reuse expert heads trained jointly with each expert's encoder, while the training-based methods learn new heads from random initialization. The observed gap therefore conflates encoder-level and head-level differences. Freezing the expert heads and using them for the training-based methods in training and evaluation would have isolated the encoder-level comparison; we did not implement this control. This confound is specific to encoder-only architectures with task-specific heads and would not arise in decoder-only models, where merging operates on all parameters jointly.

Binary classification restriction. We restrict to binary classification tasks to maintain uniform head dimensions across tasks. This excludes MNLI (3 classes, 393k samples), the flagship GLUE task and the one most likely to exhibit strong interference due to its size and complexity. Binary tasks may be structurally more similar, potentially biasing toward favorable merging performance. Our results therefore represent a *conservative* estimate of the gap between $\mathrm{PF}(\Theta_{\mathrm{conv}})$ and $\mathrm{PF}(\Theta)$: including multi-class tasks with more diverse head structures could widen the observed gap.

Uncontrolled fine-tuning of expert models. The TextAttack experts were fine-tuned with hyperparameters we did not select or optimize. The quality of the task vectors τ^t_{enc} depends on the fine-tuning procedure: under-trained experts produce short task vectors that may not capture the full task-specific information, while over-trained experts may produce noisy or overfit task vectors. We report the TextAttack hyperparameters in Appendix B and note that they follow standard practice, but we cannot rule out that different fine-tuning choices would yield different merging outcomes.

Limited variance estimation. For the four-task benchmark, all training-based runs use a single random seed. Since both methods converge to local optima that depend on initialization, a single seed cannot distinguish genuine Pareto front differences from initialization artifacts. The two-task benchmark uses four seeds, providing variance estimates for the primary comparison. Extending multi-seed runs to the four-task benchmark would strengthen the statistical basis of our conclusions.

Optimization error in training-based methods. The measured gap between the merging methods and the training-based methods is a lower bound on the true gap between the best attainable front in Θ_{conv} and $\mathrm{PF}(\Theta)$: this

follows directly from the fact that any realized solution can only overestimate the infimum over Θ. The calibration experiment (subsection 3.2.1) provides indirect evidence on the tightness of this bound by comparing the training-based methods to independently fine-tuned experts, but since neither achieves the global optimum, it cannot quantify the remaining gap to $\mathrm{PF}(\Theta)$.

Small evaluation sets. The MRPC test split contains only 204 samples, yielding confidence intervals of approximately ± 5 percentage points. Any observed performance differences on MRPC smaller than this margin are not statistically meaningful. We flag all such cases in the results.

Sparse preference grid (four-task benchmark). The seven preference vectors in the four-task benchmark sample the 3-simplex sparsely. Pareto front regions between the sampled preferences remain unexplored. The two-task benchmark, with nine uniformly spaced preferences on a 1-simplex, provides a denser and more reliable characterization.

Chapter 4

Conclusion and Future Work

4.1 Conclusion

This work asked whether the convex hull of (preprocessed) task vectors, the model class searched by a large family of merging methods, incurs a significant approximation error relative to the full parameter space. We compared Task Arithmetic and TIES-Merging against two structurally different training-based multi-objective methods (augmented Chebyshev scalarization and EPO Search) on binary classification tasks from the GLUE benchmark using RoBERTa-base.

Our experiments provide consistent evidence that Θ_{conv} excludes trade-offs accessible to gradient-based optimization over Θ. In the four-task benchmark, Task Arithmetic suffered catastrophic degradation on CoLA and QNLI under most preference vectors, with CoLA MCC dropping to zero at two operating points. TIES-Merging substantially mitigated this interference but retained a clear gap to both training-based methods, which maintained jointly high performance across all tasks and preferences. The two-task benchmark (MRPC–QNLI) confirmed this pattern at higher resolution: a dense preference sweep showed that the TIES Pareto front is dominated by the training-based fronts at most operating points, with the gap exceeding seed-to-seed variance. Notably, the training-based methods exhibited only mild trade-offs between tasks, consistent with RoBERTa-base possessing sufficient capacity to accommodate all four binary tasks simultaneously and with the theoretical expectation that Pareto fronts in MTL settings can collapse toward the utopia point (Remark 2.3).

The gap we observe is structural rather than algorithmic: since the merging methods were evaluated on a dense grid over the simplex, no alternative choice of merging coefficients $\boldsymbol{\alpha}$ can close it. The limitation lies in the geometry of Θ_{conv} itself, an at most $(T-1)$-dimensional subset of a space with over 125 million dimensions. When encoder-level task vectors interfere

destructively, as evidenced by the cross-evaluation matrix (Table 3.2), this low-dimensional set simply does not contain parameter configurations that perform well on all tasks simultaneously.

These conclusions are subject to the limitations discussed in section 3.3. Most importantly, all experiments use a single encoder-only architecture on English binary classification tasks, while the modern merging literature focuses on decoder-only language models, vision-language models, and substantially larger scales where loss landscape properties may differ. The calibration experiment further revealed a non-trivial optimization deficit on CoLA for both training-based methods, meaning that the measured gap is a lower bound on the true restriction error whose tightness is uncertain for this task. Our findings should therefore be understood as evidence that the convex restriction *can* incur significant error under destructive interference, rather than as a general quantification of that error across architectures and task domains.

4.2 Future Work

We outline several directions that extend the scope of the present study.

Alignment and multi-objective learning. Our experiments operate in an MTL setting where the tasks have disjoint input supports and the Pareto front can in principle collapse to a single point (Remark 2.3). The most practically relevant application of model merging in large language models, however, is the post-training alignment pipeline, where expert models are obtained through reinforcement learning from human feedback (RLHF) [60] or direct preference optimization (DPO) [61], each optimizing for distinct objectives such as helpfulness, harmlessness, or honesty. This setting is fundamentally MOL rather than MTL: a single input distribution is evaluated under multiple objectives simultaneously, the disjoint support assumption does not hold, and the resulting Pareto front is non-degenerate regardless of model capacity. [44] demonstrated that interpolating models fine-tuned on diverse reward functions can navigate such trade-offs, but whether Θ_{conv} captures the achievable Pareto front in this setting, where objective conflict is genuine rather than an artifact of limited capacity, remains untested. Additionally, this setting differs architecturally: there are no separate classification heads, so merging operates on all parameters jointly, eliminating the encoder-versus-head decomposition from our experiment. Furthermore, modern fine-tuning settings frequently use low-rank adaptation (LoRA) [62] rather than full-parameter updates, which introduces additional challenges for merging. Recent work has shown that LoRA fine-tuned models exhibit misaligned update subspaces across tasks, causing standard merging methods to degrade significantly compared to their performance on full-rank

fine-tuned models [22]. Whether our findings on the limitation of Θ_{conv} extend to the low-rank regime, and whether the approximation error is amplified or attenuated by the additional subspace misalignment, is an open question.

Scaling to more expert models. Our experiments merge at most four expert models, but community platforms host dozens to hundreds of fine-tuned variants of popular base models. Empirical and theoretical evidence suggests diminishing returns: [63] show that performance saturates after approximately ten experts due to constraints imposed by the effective parameter space. Understanding how the gap between $\text{PF}(\Theta_{\text{conv}})$ and $\text{PF}(\Theta)$ evolves as T increases, and whether training-based methods are subject to similar saturation, is an open question that our framework could address.

Broader method comparison. We evaluated two merging methods (Task Arithmetic and TIES-Merging) and two training-based methods (Chebyshev scalarization and EPO Search). The merging literature offers a substantially larger set of data-free methods, including DARE [45] and PCB-Merging [46], as well as data-aware methods such as Fisher-weighted merging [10] and AdaMerging [14]. These methods define different effective parameter sets through alternative preprocessing heuristics or coefficient selection strategies. A broader comparison would clarify whether the gap we observe is specific to Task Arithmetic and TIES-Merging or reflects a more general limitation of the convex merging paradigm.

Beyond convex combinations of task vectors. Our results suggest that expanding the effective search space beyond Θ_{conv} is necessary to close the gap with training-based methods. However, the practical value of data-free merging lies not only in finding a single good model but in efficiently producing a *family* of models, one for each user-specified preference, at negligible marginal cost. If no training pipeline interventions are implemented [13], this steerability requirement constrains the design of non-convex alternatives: the method must efficiently encode an entire region of the Pareto set, not just a single point. Hypernetworks [24] and preference-conditioned generators [25] can, in principle, represent non-convex regions of the Pareto set by amortizing multi-objective optimization into a single upfront training phase. However, it is not clear how to parameterize such mappings so that they efficiently encode the relevant non-convex structure in high-dimensional parameter spaces while remaining tractable to train and faithful across the full preference simplex. The task vector framework succeeds in part because it provides a natural, interpretable, low-dimensional parameterization; replacing it with a more expressive but less structured search space introduces challenges in optimization, generalization, and interpretability.

Identifying principled intermediate representations between the $(T-1)$-dimensional simplex and the full parameter space Θ that preserve efficient steerability is, in our view, a central open problem for the field.

Bibliography

[1] WX Zhao et al. *A Survey of Large Language Models*. arXiv:2303.18223 [cs]. 03/2026. DOI: 10.48550/arXiv.2303.18223.

[2] A Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. arXiv:2103.00020 [cs]. 02/2021. DOI: 10.48550/arXiv.2103.00020.

[3] J Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J Burstein, C Doran, and T Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, 06/2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.

[4] C Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.

[5] B McMahan et al. "Communication-Efficient Learning of Deep Networks from Decentralized Data". en. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*. PMLR, 04/2017, pp. 1273–1282.

[6] N Carlini et al. "Extracting Training Data from Large Language Models". en. In: 2021, pp. 2633–2650.

[7] D Marczak et al. "MAGMAX: Leveraging Model Merging for Seamless Continual Learning". en. In: *Computer Vision – ECCV 2024*. Ed. by A Leonardis et al. Cham: Springer Nature Switzerland, 2025, pp. 379–395. DOI: 10.1007/978-3-031-73013-9_22.

[8] Y Li et al. *Model Merging in Pre-training of Large Language Models*. arXiv:2505.12082 [cs]. 05/2025. DOI: 10.48550/arXiv.2505.12082.

[9] E Yang et al. "Model Merging in LLMs, MLLMs, and Beyond: Methods, Theories, Applications, and Opportunities". en. In: *ACM Computing Surveys* 58.8 (06/2026), pp. 1–41. DOI: 10.1145/3787849.

[10] MS Matena and CA Raffel. "Merging Models with Fisher-Weighted Averaging". en. In: *Advances in Neural Information Processing Systems* 35 (12/2022), pp. 17703–17716.

[11] G Ilharco et al. "Editing models with task arithmetic". en. In: 09/2022.

[12] P Yadav et al. "TIES-Merging: Resolving Interference When Merging Models". en. In: *Advances in Neural Information Processing Systems* 36 (12/2023), pp. 7093–7115.

[13] K Wang et al. "Conditional Language Policy: A General Framework For Steerable Multi-Objective Finetuning". In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Y Al-Onaizan, M Bansal, and YN Chen. Miami, Florida, USA: Association for Computational Linguistics, 11/2024, pp. 2153–2186. DOI: 10.18653/v1/2024.findings-emnlp.118.

[14] E Yang et al. "AdaMerging: Adaptive Model Merging for Multi-Task Learning". en. In: arXiv, 10/2023.

[15] R Caruana. "Multitask Learning". en. In: *Machine Learning* 28.1 (07/1997), pp. 41–75. DOI: 10.1023/A:1007379606734.

[16] M Crawshaw. *Multi-Task Learning with Deep Neural Networks: A Survey*. arXiv:2009.09796 [cs]. 09/2020. DOI: 10.48550/arXiv.2009.09796.

[17] O Sener and V Koltun. "Multi-Task Learning as Multi-Objective Optimization". In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018.

[18] M Ruchte and J Grabocka. *Multi-task problems are not multi-objective*. arXiv:2110.07301 [cs]. 10/2021. DOI: 10.48550/arXiv.2110.07301.

[19] SK Ainsworth, J Hayase, and S Srinivasa. *Git Re-Basin: Merging Models modulo Permutation Symmetries*. arXiv:2209.04836 [cs]. 03/2023. DOI: 10.48550/arXiv.2209.04836.

[20] K Zhan, P Li, and L Wu. *Analyzing the Role of Permutation Invariance in Linear Mode Connectivity*. arXiv:2503.06001 [stat]. 03/2025. DOI: 10.48550/arXiv.2503.06001.

[21] W Ruan et al. *From Task-Specific Models to Unified Systems: A Review of Model Merging Approaches*. arXiv:2503.08998 [cs]. 03/2025. DOI: 10.48550/arXiv.2503.08998.

[22] H Zhang and J Zhou. "Unraveling LoRA Interference: Orthogonal Subspaces for Robust Model Merging". In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by W Che et al. Vienna, Austria: Association for Computational Linguistics, 07/2025, pp. 26459–26472. DOI: 10.18653/v1/2025.acl-long.1284.

[23] G Ortiz-Jimenez, A Favero, and P Frossard. "Task Arithmetic in the Tangent Space: Improved Editing of Pre-Trained Models". en. In: *Advances in Neural Information Processing Systems* 36 (12/2023). arXiv: 2305.12827 CorpusID: 258832777, pp. 66727–66754. DOI: 10.48550/arXiv.2305.12827.

[24] A Navon et al. "Learning the Pareto Front with Hypernetworks". en. In: 10/2020.

[25] W Chen and J Kwok. "Pareto Merging: Multi-Objective Optimization for Preference-Aware Model Merging". en. In: *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 10/2025, pp. 8345–8363.

[26] B Lakshminarayanan, A Pritzel, and C Blundell. "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017.

[27] M Wortsman et al. "Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time". en. In: *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 06/2022, pp. 23965–23998.

[28] DH Wolpert. "Stacked generalization". en. In: *Neural Networks* 5.2 (01/1992), pp. 241–259. DOI: 10.1016/S0893-6080(05)80023-1.

[29] RA Jacobs et al. "Adaptive Mixtures of Local Experts". en. In: *Neural Computation* 3.1 (02/1991), pp. 79–87. DOI: 10.1162/neco.1991.3.1.79.

[30] N Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. arXiv:1701.06538 [cs]. 01/2017. DOI: 10.48550/arXiv.1701.06538.

[31] W Fedus, B Zoph, and N Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. arXiv:2101.03961 [cs]. 06/2022. DOI: 10.48550/arXiv.2101.03961.

[32] M Muqeeth, H Liu, and C Raffel. "Soft Merging of Experts with Adaptive Routing". en. In: *Transactions on Machine Learning Research* (01/2024).

[33] G Hinton, O Vinyals, and J Dean. *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531 [stat]. 03/2015. DOI: 10.48550/arXiv.1503.02531.

[34] S You et al. "Learning from Multiple Teacher Networks". en. In: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Halifax NS Canada: ACM, 08/2017, pp. 1285–1294. DOI: 10.1145/3097983.3098135.

[35] V Sanh et al. *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv:1910.01108 [cs]. 03/2020. DOI: 10.48550/arXiv.1910.01108.

[36] D Ha, A Dai, and QV Le. *HyperNetworks*. arXiv:1609.09106 [cs]. 12/2016. DOI: 10.48550/arXiv.1609.09106.

[37] S Han et al. *Learning both Weights and Connections for Efficient Neural Networks*. arXiv:1506.02626 [cs]. 10/2015. DOI: 10.48550/arXiv.1506.02626.

[38] K Liu et al. *Low-bit Model Quantization for Deep Neural Networks: A Survey*. arXiv:2505.05530 [cs]. 05/2025. DOI: 10.48550/arXiv.2505.05530.

[39] K Miettinen. *Nonlinear Multiobjective Optimization*. eng. 1st ed. International Series in Operations Research and Management Science Ser v.12. Boston: Springer, 1998.

[40] D Mahapatra and V Rajan. "Multi-Task Learning with User Preferences: Gradient Descent with Controlled Ascent in Pareto Optimization". en. In: *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 11/2020, pp. 6597–6607.

[41] JA Désidéri. "Multiple-gradient descent algorithm (MGDA) for multiobjective optimization". en. In: *Comptes Rendus. Mathématique* 350.5-6 (03/2012), pp. 313–318. DOI: 10.1016/j.crma.2012.03.014.

[42] X Lin et al. *Pareto Multi-Task Learning*. arXiv:1912.12854 [cs]. 12/2019. DOI: 10.48550/arXiv.1912.12854.

[43] SC Huang et al. "Chat Vector: A Simple Approach to Equip LLMs with Instruction Following and Model Alignment in New Languages". en. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 10943–10959. DOI: 10.18653/v1/2024.acl-long.590.

[44] A Ramé et al. *Rewarded soups: towards Pareto-optimal alignment by interpolating weights fine-tuned on diverse rewards*. arXiv:2306.04488 [cs]. 10/2023. DOI: 10.48550/arXiv.2306.04488.

[45] L Yu et al. "Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch". en. In: *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 07/2024, pp. 57755–57775.

[46] G Du et al. *Parameter Competition Balancing for Model Merging*. arXiv:2410.02396 [cs]. 10/2024. DOI: 10.48550/arXiv.2410.02396.

[47] M Davari and E Belilovsky. "Model Breadcrumbs: Scaling Multi-task Model Merging with Sparse Masks". en. In: *Computer Vision – ECCV 2024*. Ed. by A Leonardis et al. Cham: Springer Nature Switzerland, 2025, pp. 270–287. DOI: 10.1007/978-3-031-73226-3_16.

[48] Z Wang et al. *Scaling Intelligence Through Model Merging: A Comprehensive Survey*. 11/2025. DOI: 10.36227/techrxiv.176315557.76673270/v1.

[49] T Garipov et al. "Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs". In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018.

[50] F Draxler et al. *Essentially No Barriers in Neural Network Energy Landscape*. arXiv:1803.00885 [stat]. 02/2019. DOI: 10.48550/arXiv.1803.00885.

[51] J Frankle et al. "Linear Mode Connectivity and the Lottery Ticket Hypothesis". en. In: *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 11/2020, pp. 3259–3269.

[52] Z Zhou et al. *On the Emergence of Cross-Task Linearity in the Pretraining-Finetuning Paradigm*. en. arXiv:2402.03660 [cs]. 05/2024. DOI: 10.48550/arXiv.2402.03660.

[53] G Benton et al. "Loss Surface Simplexes for Mode Connecting Volumes and Fast Ensembling". en. In: *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 07/2021, pp. 769–779.

[54] C Li et al. *Measuring the Intrinsic Dimension of Objective Landscapes*. arXiv:1804.08838 [cs]. 04/2018. DOI: 10.48550/arXiv.1804.08838.

[55] A Aghajanyan, S Gupta, and L Zettlemoyer. "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning". en. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Online: Association for Computational Linguistics, 2021, pp. 7319–7328. DOI: 10.18653/v1/2021.acl-long.568.

[56] Y Liu et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692 [cs]. 07/2019. DOI: 10.48550/arXiv.1907.11692.

[57] A Wang et al. "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding". en. In: *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 353–355. DOI: 10.18653/v1/W18-5446.

[58] J Morris et al. "TextAttack: A Framework for Adversarial Attacks, Data Augmentation, and Adversarial Training in NLP". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Ed. by Q Liu and D Schlangen. Online: Association for Computational Linguistics, 10/2020, pp. 119–126. DOI: 10.18653/v1/2020.emnlp-demos.16.

[59] I Loshchilov and F Hutter. *Decoupled Weight Decay Regularization*. arXiv:1711.05101 [cs]. 01/2019. DOI: 10.48550/arXiv.1711.05101.

[60] S Agarwal et al. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems 35*. New Orleans, Louisiana, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022, pp. 27730–27744. DOI: 10.52202/068431-2011.

[61] S Ermon et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems 36*. New Orleans, Louisiana, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 53728–53741. DOI: 10.52202/075280-2338.

[62] EJ Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685 [cs]. 10/2021. DOI: 10.48550/arXiv.2106.09685.

[63] Z Wang et al. *Why Do More Experts Fail? A Theoretical Analysis of Model Merging*. arXiv:2505.21226 [cs]. 06/2025. DOI: 10.48550/arXiv.2505.21226.

Appendices

Appendix A

MOO formalizations

A.1 Pathological MOL example

We construct a minimal multi-objective learning instance in which the Pareto front is non-degenerate regardless of model capacity, showing that the collapse mechanism described in Remark 2.3 is specific to the MTL setting and does not extend to MOL.

Setup. Let $\mathcal{X} = \{x_0\}$ be a singleton input space, $\mathcal{Y} = \mathbb{R}$ the output space, and $P = \delta_{(x_0,0)}$ the distribution placing all mass on the pair $(x_0, 0)$. Define two loss functions $\ell^1, \ell^2 \colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$ by

$$\ell^1(\hat{y}, y) = (\hat{y} - y)^2, \qquad \ell^2(\hat{y}, y) = (\hat{y} - y - 1)^2.$$

This is an MOL instance: the single input x_0 is evaluated under both ℓ^1 and ℓ^2 simultaneously, and the marginal input supports coincide ($\mathrm{Supp}(P^1_{\mathcal{X}}) = \mathrm{Supp}(P^2_{\mathcal{X}}) = \{x_0\}$), so the disjoint support assumption of Remark 2.3 is violated.

Since $P = \delta_{(x_0,0)}$, the population risks from section 2.1 reduce for any model $f(\theta, \cdot)$ to

$$\mathcal{R}^1(\theta) = \mathbb{E}_{(X,Y)\sim P}\big[\ell^1\big(f(\theta, X), Y\big)\big] = \ell^1\big(f(\theta, x_0), 0\big) = f(\theta, x_0)^2$$

and $\mathcal{R}^2(\theta) = (f(\theta, x_0) - 1)^2$ analogously. Both risks depend on θ only through the scalar output $y(\theta) := f(\theta, x_0) \in \mathcal{Y}$. The achievable risk set is therefore

$$\mathbf{R}(\Theta) = \big\{\big(\mathcal{R}^1(\theta), \mathcal{R}^2(\theta)\big) : \theta \in \Theta\big\} = \{(y(\theta)^2, (y(\theta) - 1)^2) : \theta \in \Theta\}.$$

Claim. The utopia point $\mathbf{z}^\star = (0, 0)$ is not attainable, and $\mathrm{PF}(\Theta)$ is a non-degenerate curve. This holds for any parameter space Θ and any model f such that $\{f(\theta, x_0) : \theta \in \Theta\} \supseteq [0, 1]$.

Proof. Attaining $\mathbf{z}^\star$ requires some $\theta^\star \in \Theta$ with $y(\theta^\star)^2 = 0$ and $(y(\theta^\star) - 1)^2 = 0$, i.e., $y(\theta^\star) = 0$ and $y(\theta^\star) = 1$ simultaneously. Since $y(\theta^\star)$ is a single real number, this is a contradiction.

By the range assumption, for each $y \in [0,1]$ there exists $\theta \in \Theta$ with $y(\theta) = y$. Since both risks depend on θ only through $y(\theta)$, we write $\mathcal{R}^t(y)$ for $\mathcal{R}^t(\theta)$ when $y = y(\theta)$.

It remains to show that every $y \in [0,1]$ is Pareto optimal and every $y \notin [0,1]$ is strictly dominated. Note that $\ell^1(\cdot, 0) = (\cdot)^2$ is strictly increasing on $[0,\infty)$ and $\ell^2(\cdot, 0) = (\cdot - 1)^2$ is strictly decreasing on $(-\infty, 1]$. For any $y, y' \in [0,1]$ with $y < y'$, we have $\mathcal{R}^1(y) < \mathcal{R}^1(y')$ and $\mathcal{R}^2(y) > \mathcal{R}^2(y')$, so neither risk vector dominates the other (Definition 2.1). Hence every $y \in [0,1]$ is Pareto optimal.

For $y < 0$, compare with $y' = 0$: $\mathcal{R}^1(0) = 0 < y^2 = \mathcal{R}^1(y)$ and $\mathcal{R}^2(0) = 1 < (y-1)^2 = \mathcal{R}^2(y)$, so $y' = 0$ strictly dominates y. The case $y > 1$ is symmetric, with $y' = 1$ strictly dominating y. The Pareto front is therefore

$$\mathrm{PF}(\Theta) = \left\{ \left(y^2,\, (y-1)^2\right) : y \in [0,1] \right\},$$

a non-degenerate curve connecting $(0,1)$ to $(1,0)$ in $\mathbb{R}^2_{\geq 0}$ (Figure A.1).

Discussion. The non-degeneracy is a consequence of the MOL structure: the model must commit to a single output for the shared input x_0, and that output cannot simultaneously minimize two losses with distinct minimizers. No increase in model capacity, parameter dimension, or optimization effort can resolve this conflict.

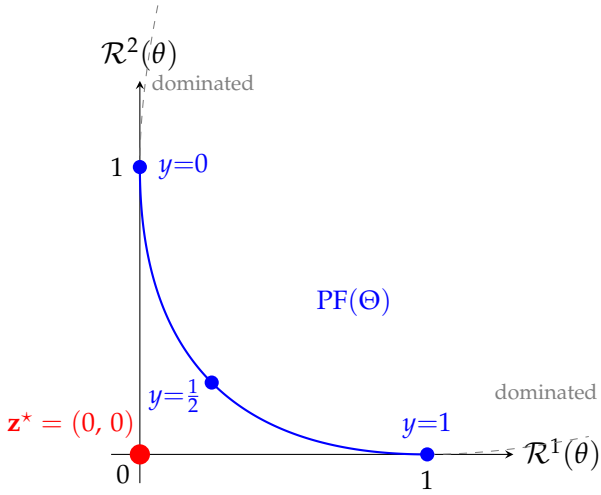

Figure A.1: Pareto front of the MOL example. The solid curve traces $\mathrm{PF}(\Theta) = \{(y^2, (y-1)^2) : y \in [0,1]\}$. The dashed extensions correspond to outputs $y \notin [0,1]$, which are strictly dominated. The utopia point $\mathbf{z}^\star = (0,0)$ coincides with the origin and is unattainable because no single output y simultaneously minimizes both losses.

A.2 Augmented Weighted Chebyshev Scalarization: Proof and Discussion

Proof of Theorem 2.8

We restate the result for convenience.

Let $\Theta' \subseteq \Theta$, $\mathbf{w} \in \Delta^{T-1}$, $\rho > 0$, and $\mathbf{z} \in \mathbb{R}^T$ with $z_t \leq \mathcal{R}^t(\theta)$ for all $t \in [T]$ and $\theta \in \Theta'$. Then every minimizer of

$$L(\theta) = \max_{t \in [T]} w_t\big(\mathcal{R}^t(\theta) - z_t\big) + \rho \sum_{t=1}^{T} w_t\big(\mathcal{R}^t(\theta) - z_t\big)$$

over Θ' is Pareto optimal in Θ'.

Proof Suppose for contradiction that $\theta^\star$ minimizes L over Θ' but is not Pareto optimal. By Definition 2.2, there exists $\theta \in \Theta'$ with $\mathcal{R}^t(\theta) \leq \mathcal{R}^t(\theta^\star)$ for all $t \in [T]$ and $\mathcal{R}^s(\theta) < \mathcal{R}^s(\theta^\star)$ for some $s \in [T]$.

The componentwise inequality and $w_t > 0$ give $w_t(\mathcal{R}^t(\theta) - z_t) \leq w_t(\mathcal{R}^t(\theta^\star) - z_t)$ for every t. Taking the maximum preserves the weak inequality:

$$\max_{t \in [T]} w_t\big(\mathcal{R}^t(\theta) - z_t\big) \leq \max_{t \in [T]} w_t\big(\mathcal{R}^t(\theta^\star) - z_t\big). \tag{A.1}$$

For the augmentation term, the inequality is strict at $t = s$ (since $w_s > 0$ and $\mathcal{R}^s(\theta) < \mathcal{R}^s(\theta^\star)$) and weak at all other indices, so the sum is strictly smaller:

$$\rho \sum_{t=1}^{T} w_t\big(\mathcal{R}^t(\theta) - z_t\big) < \rho \sum_{t=1}^{T} w_t\big(\mathcal{R}^t(\theta^\star) - z_t\big). \tag{A.2}$$

Adding (A.1) and (A.2) gives $L(\theta) < L(\theta^\star)$, contradicting minimality. □

Discussion: Proper Pareto Optimality and the Converse Direction

Define transformed objectives $g_t(\theta) = w_t(\mathcal{R}^t(\theta) - z_t)$. The augmented Chebyshev objective becomes $L(\theta) = \max_t g_t(\theta) + \rho \sum_t g_t(\theta)$, which is an instance of Miettinen's Problem (3.4.5) [39] applied to $(g_1, \ldots, g_T)$ with uniform Chebyshev weights and reference point $\tilde{\mathbf{z}} = \mathbf{0}$. Since $w_t > 0$, the Pareto ordering under $\mathbf{g}$ coincides with that under $\mathbf{R}$.

This reduction has two consequences. First, [39, Part II, Theorem 3.4.6] implies that every minimizer is in fact properly Pareto optimal (in the sense of [39, Part I, Definition 2.9.1]), meaning the trade-off ratios between objectives are bounded. The bound under $\mathbf{R}$ is $M_g \cdot (\max_j w_j)/(\min_j w_j)$, where M_g is the bound under $\mathbf{g}$. This is strictly stronger than the Pareto optimality stated in Proposition 2.8.

Second, for the converse direction, Theorem 3.4.6 guarantees that every properly Pareto optimal point solves Problem (3.4.5) for some weight vector $\tilde{\mathbf{w}} > 0$ and sufficiently small ρ. Applied to the transformed objectives, this gives $\theta^\star$ as a minimizer of $\max_t \tilde{w}_t g_t(\theta) + \rho \sum_t g_t(\theta)$. Substituting $g_t = w_t(\mathcal{R}^t - z_t)$ yields Chebyshev weights $\tilde{w}_t w_t$ paired with augmentation weights w_t. Our coupled formulation requires these to be proportional, i.e., $\tilde{w}_t$ must be constant across t. Therefore, the converse direction of the theorem does not apply directly and the proof must be adapted to our setting (Definition 2.7).

Appendix B

Experiment Hyperparameters

Table B.1: Hyperparameters for the fine-tuning methods (Chebyshev scalarization and EPO). The 4-task GLUE-2-LABEL) and 2-task (MRPC/QNLI sweep) benchmarks share identical configurations; the 2-task sweep additionally uses seeds $\{100, 200, 300\}$ with all other hyperparameters unchanged. Calibration runs follow the Hyperparameters of the TextAttack models.

	Main benchmarks		**Recovery**	
Hyperparameter	**Chebyshev**	**EPO**	**Chebyshev**	**EPO**
Optimizer	AdamW			
LR schedule	linear warmup + linear decay			
Learning rate	2×10^{-5}			
Warmup steps	100		0	
Batch size	32			
Max. epochs	6		5	
Early stopping (patience)	2		–	
Max. samples per task	6,000		full data	
Mixed precision (FP16)	✓		×	
Weight decay	0.01		0	
Augmentation weight ρ (Cheby.)	0.001	–	0.001	–
Seed	42			

Table B.2: Hyperparameters for TIES-Merging. Weighted averaging (task arithmetic) has no hyperparameters

Hyperparameter	**TIES-Merging**
Sparsification ratio k	0.2
Merge coefficient λ	1.0
Scaling method	magnitude
Sign consensus	weighted

Appendix C

Additional results

We report the numeric results of the two task benchmark (subsection 3.2.3). The relevant four task benchmark results are apparent from Figure 3.1.

Table C.1: Per-preference-vector accuracy (%) on the 2-task benchmark (QNLI–MRPC). Values show the mean across seeds; $\pm$ denotes one standard deviation (omitted for single-seed runs).

Pref. vector	Averaging		TIES		Chebyshev		EPO	
	QNLI	MRPC	QNLI	MRPC	QNLI	MRPC	QNLI	MRPC
(.9, .1)	54.6	**89.7**	80.3	86.8	**86.1 $\pm$ 2.4**	87.9 $\pm$ 1.4	79.6 $\pm$ 2.7	86.8 $\pm$ 2.0
(.8, .2)	65.2	**88.7**	80.5	86.3	**88.0 $\pm$ 1.2**	87.4 $\pm$ 1.0	80.9 $\pm$ 1.9	88.1 $\pm$ 1.6
(.7, .3)	76.3	86.8	81.4	86.3	**88.9 $\pm$ 0.7**	**88.4 $\pm$ 1.9**	84.5 $\pm$ 0.8	87.6 $\pm$ 3.2
(.6, .4)	82.1	85.8	83.8	**86.8**	**89.0 $\pm$ 0.5**	86.6 $\pm$ 1.5	82.0 $\pm$ 1.1	86.6 $\pm$ 1.9
(.5, .5)	86.0	79.4	85.6	85.3	**89.4 $\pm$ 0.6**	86.9 $\pm$ 1.1	87.0 $\pm$ 0.4	**88.1 $\pm$ 2.5**
(.4, .6)	**89.5**	76.0	86.5	84.8	89.4 $\pm$ 0.7	88.2 $\pm$ 0.8	88.3 $\pm$ 0.7	**89.5 $\pm$ 0.7**
(.3, .7)	**91.5**	70.6	87.0	84.8	89.6 $\pm$ 0.3	86.3 $\pm$ 1.8	89.6 $\pm$ 0.4	**87.6 $\pm$ 1.2**
(.2, .8)	**92.1**	68.1	87.1	84.8	89.4 $\pm$ 0.5	85.0 $\pm$ 1.4	89.6 $\pm$ 0.5	**86.3 $\pm$ 2.5**
(.1, .9)	**92.2**	68.1	87.1	84.8	89.6 $\pm$ 0.8	**87.3 $\pm$ 0.0**	89.5 $\pm$ 0.2	85.0 $\pm$ 1.7